

UAS FIDO2 Token Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAS FIDO2 Token Implementation Guide.....	3
2. Introduction to UAS FIDO2 Token Implementation.....	4
3. UAS FIDO2 Concepts.....	9
4. Pre-Registered FIDO2 Tokens.....	11
5. UAS FIDO2 Token Configuration.....	21
6. UAS FIDO2 Token Implementation Using API.....	38
6.1. Token Service.....	39
6.1.1. Token Registration.....	40
6.1.2. Token Assertion.....	47
6.1.3. Token Verification.....	53
6.1.4. Token Management.....	58
6.2. Authentication Service.....	69
6.2.1. Authentication Registration.....	70
6.2.2. Authentication Assertion.....	76
7. UAS FIDO2 Token Housekeeping Policy.....	82
8. UAS FIDO2 Token Reports.....	84
9. Appendices.....	85

1. Preface - UAS FIDO2 Token Implementation Guide

This document provides detailed information on the integration and configuration of FIDO2 authenticators with the AccessMatrix Universal Authentication Server (UAS). In addition, it consists of an API integration guide for FIDO2-based tokens with the AccessMatrix UAS system.

Intended Reader

This is intended as a reference for the security administrators, i-Sprint's Professional Service personnel, and developers.

Document Scope

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- [*AccessMatrix Common Administration Guide*](#)
- [*UAS Administration Guide*](#)
- [*AccessMatrix REST API Specification*](#)

2. Introduction to UAS FIDO2 Token Implementation

FIDO2 authentication is a set of open standards for secure authentication to online services using common devices. It is based on public key cryptography. The AccessMatrix UAS system supports the integration of FIDO2-based authenticators to address the strong authentication requirements of a wide range of enterprise applications. The solution provides administration, authentication, authorization, and audit services to address security and compliance requirements.

Key Benefits

- **Enhanced security with Multi-Factor Authentication**

Multi-factor authentication provides exceptionally high levels of security to protect against hacking attacks. Two-factor authentication, a type of multi-factor authentication with FIDO2-based authenticators, is used to secure network access, protect financial accounts, digitally sign transactions, and keep criminals from causing losses by preventing unauthorized network access.

- **Scalability**

You can add more users, and use the tokens to secure more and other business applications.

UAS Key Features

- **Multi-Factor Authentication**

This adds additional security measures to the standard username, password logins across the AccessMatrix UAS system that stops unauthorized logins, even the passwords that have been shared among different services.

- **Token Management**

The administrator can determine and manage the user-to-token relationships.

- **Token Grouping Management**

The administrator can configure the token batches and groups to the respective users.

- **Token Integration APIs**

FIDO2 API integration with the AccessMatrix UAS system is supported.

- **Security Auditing and Reporting**

Audit reports are available in a wide selection of PDF, HTML and XML formats.

- **Scalable and High Availability Architecture**

It supports applications that can utilize industry-standard protocols and applications along with high-availability support.

FIDO2 Integration with AccessMatrix

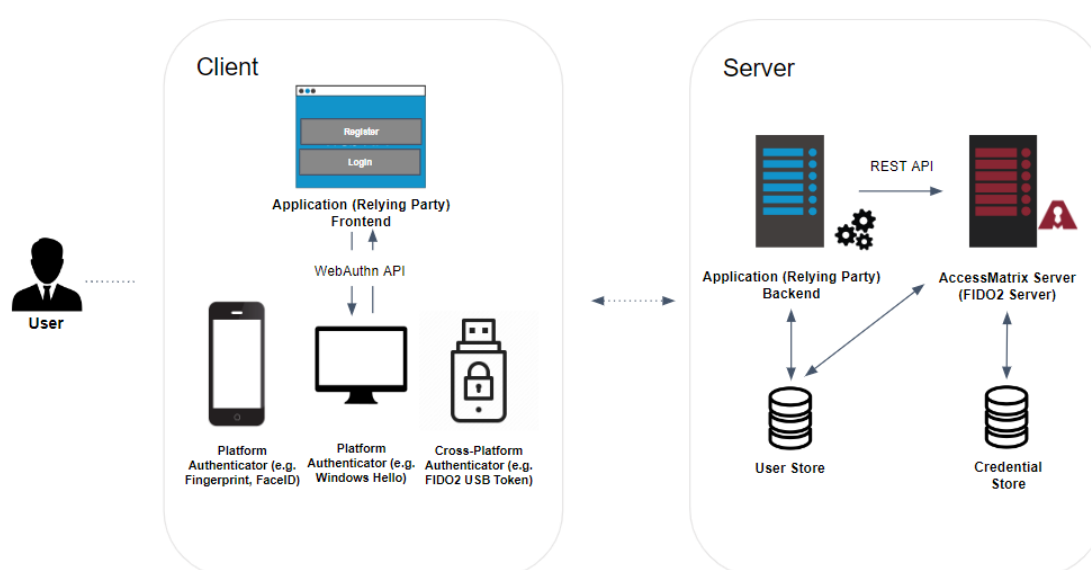
The following diagram illustrates how FIDO2 is integrated with the AccessMatrix system and a client web application.

The AccessMatrix server acts as the FIDO2 server, which stores the FIDO credentials and authenticates the end user.

The web application processes the user's authentication request, then forwards the request to the AccessMatrix server via API and relies on it to perform the user authentication. The web application (both its front end and back end, collectively) is also referred to as the Relying Party in the AccessMatrix Admin Console and in this guide.

Any FIDO2-compliant authenticator can be used by the user for FIDO2 authentication.

AccessMatrix FIDO2 Integration



Manage FIDO2 Token Life Cycle

AccessMatrix supports multiple authentication methods to enable organizations to deploy the appropriate level of authentication technologies to address security requirements. The following section describes how you can manage the life cycle of FIDO2 tokens.

Token Management

Manage different models of FIDO2 tokens by carrying out the following:

- **Token Assignment**

Users are able to self-register and deregister FIDO2 authenticators.

- **Token Re-Sync**

Token may be out-of-sync with the security server due to the time drift from the GMT. To resolve this problem, the user's token needs to be resynchronized.

Implementation Checklist

Token(s)	Reference
Configure token policy	Configure FIDO2 Token Policy
Configure token batch	Configure Token Batch
Configure token group	Configure Token Group
User(s)	Reference
(Optional) Configure lookup module	Configure Lookup Module
Configure login module	Configure Login Module
Configure authentication realm	Configure Authentication Realm
Configure user accounts	Configure User Accounts

Implementation Planning

During implementation, you need to understand the customer's nature of business, studying the customer's requirements and its existing environments to carry out the action plans. The following discuss the security and integration considerations.

Security Consideration

The token policy set by the administrator defines the behavior of authentication and authorization for security purposes. The administrator needs to identify and set the token policy in the admin console.

For FIDO2 authenticators, the token policy implementation "FIDO2 Token Policy" contains the parameters required for them to work with UAS. You should consider the following:

- Determine if your implementation requires authenticator attestation. If attestation is required, you must obtain the following and configure them in the policy:
 - A JWT FIDO2 metadata blob.
 - The certificate used to verify the JWT FIDO2 metadata blob.
 - The JSON metadata statement file of the FIDO2 authenticator being used.
- Consider creating a list of whitelisted RP IDs for additional security. This is a feature supported by UAS, configurable within the token policy.
- Consider limiting the number of authenticators allowed to be registered by a user per RP ID for additional security. This is a feature supported by UAS, configurable within the token policy.

Integration Consideration

Web Authentication (WebAuthn) is a core component of FIDO2. It is a web-based API that allows websites to update their login pages to add FIDO-based authentication on supported browsers and platforms. Browsers and platforms must be WebAuthn compatible to work with FIDO2 authenticators.

Currently, the following browsers are compatible with FIDO2 WebAuthn:

WebAuthn browser compatibility

Desktop Browser	Chrome	Firefox	Edge	Safari
Supported versions	Version 67 and above	Version 60 and above (partial support only)	Version 18 and above	Version 13 and above

WebAuthn platform compatibility

OS	Windows
Supported versions	Windows 10 22H2
	Windows 11 22H2

**Notes:**

- You should check the **Support for FIDO2: WebAuthn and CTAP** section of the [FIDO Alliance's FIDO2 webpage](#) for the latest official updates on FIDO2 WebAuthn compatibility.
- For a quick reference on WebAuthn support for browsers, you can use <https://caniuse.com/?search=webauthn>.

3. UAS FIDO2 Concepts

This section illustrates concepts that are essential to FIDO2 authentication.

Relying Party ID and Origin

Relying Party ID (RPID) and Origin are similar yet distinct concepts in FIDO2 authentication.

The RPID refers to the unique identifier of the Relying Party using FIDO2 authentication. This is typically determined from the Relying Party's domain (e.g. i-sprint.com).

Origin, on the other hand, refers to the source of the WebAuthn API call. This can be in the form of a webpage URL or application identifier. In a legitimate login, the origin refers to the Relying Party that calls the WebAuthn API.

The differences between RPID and Origin are summarized in the table below:

Aspect	Relying Party ID (RPID)	Origin
Scope	Domain level	URL-level (web) or app-level (mobile)
Example (Web)	example.com	https://login.example.com:443
Example (Mobile)	example.com	android:apk-key-hash:abcd1234 (Android) or ios:bundle-id:com.example.app (iOS)

RPID and Origin in Securing Authentication

Both RPID and Origin are essential in securing authentication in both web and mobile contexts. The interaction between the two concepts are illustrated in the following scenarios:

- [Web Context](#)
- [Mobile Context](#)

Web Context

The table below illustrates the RPID and Origin of a Relying Party and a malicious third-party in the web context. The RPID has been scoped to the Relying Party's domain, while the Origin refers to the URL calling the WebAuthn API.

Actor	RPID	Origin
Relying Party	legitbankexample.com	https://legitbankexample.com
Malicious website	iegitbankexampie.com	https://legitbankexample.com



Note: In this example, the uppercase "i" was used to make the malicious website seem genuine.

In this scenario, a malicious website attempts to use a fake login page to impersonate the Relying Party and calls the WebAuthn API. Authentication will be denied as the malicious website's RPID (https://iegitbankexampie.com) is not a valid domain of the legitimate RPID (legitbankexample.com).

Mobile Context

The table below illustrates the RPID and Origin of a Relying Party and a malicious third-party in the mobile context. The RPID remains scoped to the Relying Party's domain, while the Origin refers to the application calling the WebAuthn API.

Actor	RPID	Origin
Relying Party	example.com	android:apk-key-hash:abcd1234
Malicious application	example.com	android:apk-key-hash:fakeapp4321

The mobile platform enforces that only applications with the correct Origin can access credentials scoped to the RPID. When a malicious application attempts to use credentials scoped to **example.com**, authentication will be denied due to the mismatch between its Origin (android:apk-key-hash:fakeapp4321) and the Origin of the Relying Party (android:apk-key-hash:abcd1234).

4. Pre-Registered FIDO2 Tokens

This section describes how to integrate pre-registered FIDO2 tokens for users.

Pre-registered FIDO2 tokens are tokens that can be used without the need for manual registration. Token manufacturers provide pre-registered token credentials in a formatted file (e.g. TSV file), which can be imported into AccessMatrix UAS.

i **Note:** For more details on the integration process, refer to [Sequence Diagram for Activating Pre-Registered Tokens](#).

FIDO2 Token Assignment

When FIDO2 tokens are registered, activated, or assigned, credentials will be linked to a specific user. In each linked credential, the Unique User Identifier (userUUID) is stored within the token blob.

i **Note:** These actions correspond to `/FIDO2/registration/finish`, `/FIDO2/pre-reg/finish`, and `/FIDO2/pre-reg/assignCredential` APIs respectively.

Once the token has been assigned to a user, it must adhere to the following conditions:

- It cannot be reassigned to a different user.
- It can only be reactivated by the same user.

Token Assignment Restrictions

Restrictions can be placed on how tokens can be assigned to users. For more information on the types of restrictions that can be placed, refer to [UAS Administration Guide - Token Management - Token Restriction](#).

i **Note:** These token assignment restrictions will apply to all tokens, including non-FIDO2 tokens.

To configure token assignment restrictions, perform the following steps:

1. Navigate to **Administration Dashboard > Configuration > Global Setting** and click on the **Token Management** tab.
 2. Click the **Edit** button.
-

-
3. Select a restriction using the drop-down list corresponding to the **Token Assignment Restriction** attribute.
 4. Click the **Save** button.

FIDO2 Token Policy Consideration

Depending on your workflow for pre-registered FIDO2 tokens, the following additional configurations should be considered:

i **Note:** These are configurable attributes within FIDO2 token policy. For more information and how they can be configured, refer to [Configure FIDO2 Token Policy](#).

- [Integrity Checking](#)
- [Verification of Token File](#)

Integrity Checking

If integrity checking of the file containing the pre-registered token is not required, set the **Verify Pre-Registered Credential During Import** attribute to "false".

Verification of Token File

If verification of the file containing the pre-registered token is required, the metadata entry of the authenticator must be configured in the token policy. This can be specified using either the **FIDO2 Metadata JWT** or **FIDO2 Metadata JSON** attributes.

i **Note:** For more information on how to configure either of the attributes, refer to [Configure FIDO2 Token Policy](#).

Token Import Utility

To import pre-registered tokens, perform the following:

1. Launch the Token Import Utility (TIU).

i **Note:** For more details, refer to *UAS Administration Guide - Token Import Utility*.

2. Click **Import Token**.
 3. Select **Token Seeds Import**.
-

-
4. Specify the file to be used for token import in, by completing either of the following sub-steps (a,b):

a. **Upload local file** - Perform the following steps:

- i. Click the corresponding radio button.
- ii. Click **Choose File** and select the file to be uploaded.
- iii. Click **Upload**.

b. **Or specify server file** - Perform the following steps:

- i. Click the corresponding radio button.
- ii. Place the file in your AccessMatrix server's *temp* folder.

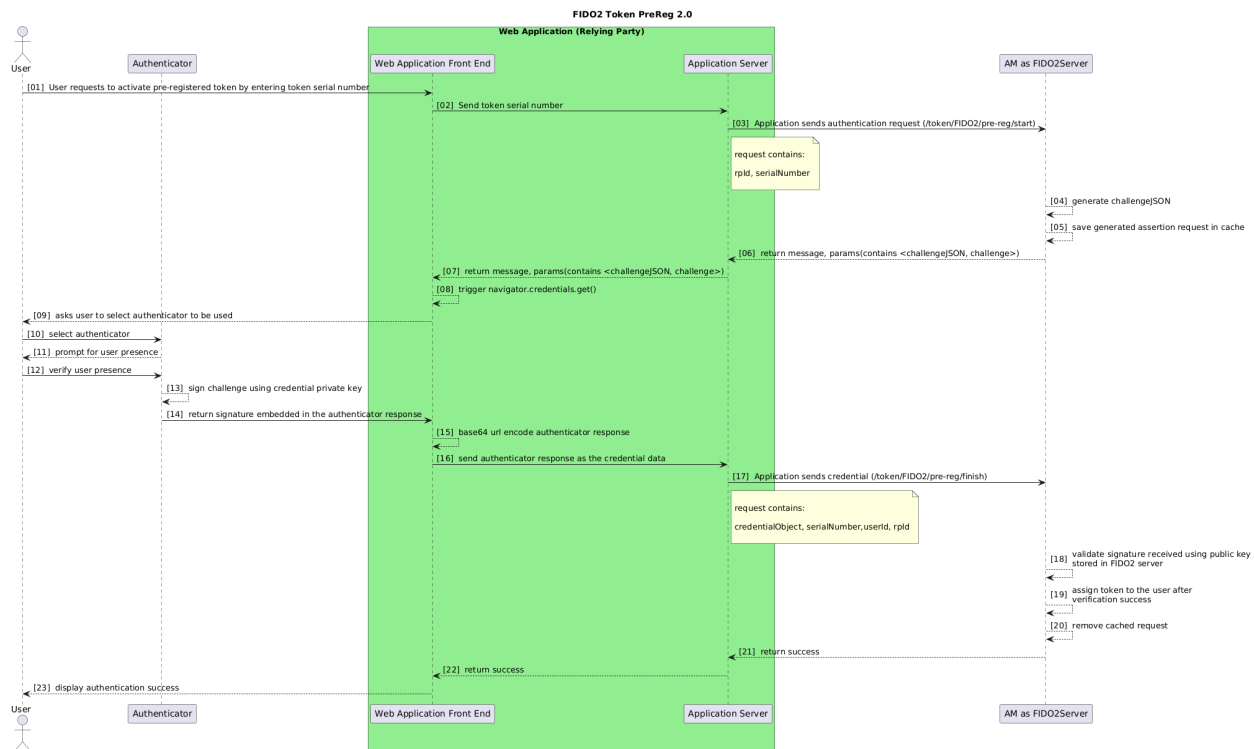
 **Note:** This folder is typically located at <ACMATRIX_ROOT>/tomcat/temp/.

- iii. Specify the name of the file.

 **Note:** You do not need to specify the path to the temp folder.

5. Select "DefaultFIDO2TokenBatch" using the drop-down list corresponding to **Token Batch**.
 6. Select "DefaultFIDO2TokenGroup" using the drop-down list corresponding to **Token Group**.
 7. Select a value using the drop-down list corresponding to Token Status.
 8. Ensure that **Issuance Token Group for System Token Assignment** is unchecked.
-  **Note:** This allows imported tokens to be assigned based on their serial number once activated via the /FIDO2/pre-reg/* APIs.
9. Specify any value for the **Transport Key** as this is not currently in use.
 10. (Optional) Click on **Preview Token** to view the tokens that will be imported. A list of tokens with their details will be displayed.
 11. Click **Import Token** to start the token import process.
-

Sequence Diagram for Activating Pre-Registered Tokens



1. The user provides the token serial number to the RP.
2. The web application calls the `/FIDO2/pre-reg/start` API.
3. The FIDO2 Server returns a challenge with a set of allowed credentials. This set should only contain 1 credential in this pre-registration use case.
4. The web application frontend should then trigger the WebAuthn API `navigator.credentials.get()` to retrieve the credential from the authenticator and generate a signature for the FIDO2 server to validate.
5. The web application calls the `/FIDO2/pre-reg/finish` API to validate the assertion signature.
6. The FIDO2 Server validates the assertion signature. If the validation is successful, it assigns the token to the specified `userId`, store the `rpld` and saves the token in the database. If the validation fails, error message is returned.

FIDO2 Token Pre-Reg Start

Use `/FIDO2/pre-reg/start` to start the activation process and generate the necessary activation parameters. These parameters will subsequently be returned to the Relying Party, where they will be used to retrieve pre-registered credentials from the authenticator.

The token can be validated against the following requirements:

i **Note:** The validation requires user information such as `userId` to be provided in the request body.

- Tokens cannot be reassigned to a different user.
- Tokens can only be reactivated by the same user.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
serialNumber	String	Required	Serial number of token.
origin	String	Optional	The origin of the WebAuthn request. If not provided, the origin will be derived from the RP ID with a default port of 443.
rpId	String	Required	An identifier for the Relying Party.
userVerification	String	Optional	Must be a value of 'preferred', 'discouraged' or 'required'. If not provided, the value 'preferred' will be used instead.
userId	String	Optional	User Id
userStoreId	String	Optional	Store ID if user is from external store
segmentId	String	Optional	When <code>userId</code> is not globally unique, segment must be specified.

REST Sample Code

FIDO2 Token Pre-Reg Start API Call
URL: <code>/FIDO2/pre-reg/start</code>
Request
JSON <pre>{ "sessionToken": "string", "serialNumber": "1234567", "rpId": "i-sprint.com", "userVerification": "preferred" }</pre>

FIDO2 Token Pre-Reg Start API Call
Response
JSON
<pre> { "result": { "message": "Successful Preauthentication.", "params": { "challengeJSON": "", "challenge": "" } } } </pre>

FIDO2 Token Pre-Registration Finish

i	Note: /FIDO2/pre-registration/finish performs the same functionality.
----------	--

Use /FIDO2/pre-reg/finish to finish the activation process.

A signature is returned as a response from the RP. This signature is verified using the public key within the pre-registered credential stored in the database. Upon successful verification, the rpId will be saved to the token and the token will be assigned to the user.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
serialNumber	String	Required	Serial number of token.
userId	String	Required	User Id
userStoreId	String	Optional	Store ID if user is from external store
segmentId	String	Optional	When userId is not globally unique, segment must be specified.
credentialObject	String	Required	Result of the successfully resolved promise, containing Base64 URL encoded attributes from the client side in JSON string format to be returned to FIDO2 server to be verified.
origin	String	Optional	The origin of the WebAuthn request. If not provided, the origin will be derived from the RP ID with a default port of 443.

Parameter	Data Type	Required/Optional	Description
rpId	String	Required	An identifier for the Relying Party.

REST Sample Code

FIDO2 Token Pre-Reg Finish API Call
URL: /FIDO2/pre-reg/finish
Request
JSON
<pre>{ "sessionToken": "string", "serialNumber": "1234567", "userId": "John", "credentialObject": "", "rpId": "i-sprint.com" }</pre>
Response
JSON
<pre>{ "result": { "params": { "tokenSerNum": "1234567", "tokenStatus": "Activated", "vendorId": "FIDO2", "userId": "John", "credentialFriendlyName": "FIDO2PreReg", "result": "success", "credentialId": "", "userUUID": "", "tokenUUID": "", "tokenModel": "FIDO2" } }, "amProcessingTimeMillis": 124 }</pre>

FIDO2 Token Pre-Reg Assign Credential

Use /FIDO2/pre-reg/assignCredential to assign a pre-registered token to a user.

This API is applicable for workflows which assign pre-registered tokens on behalf of users. With this workflow, the user is not required to activate or register the token directly, and the /FIDO2/pre-reg/* APIs

will not be used for token activation. The token with specified serial numbers will be assigned to the specified user and the rpId will be saved as a token attribute.

Once the tokens have been assigned, users can proceed to use it normally as per the other activation APIs.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
tokenUUID	String	Required	Token UUID
serialNumber	String	Required	Serial number of token.
userUUID	String	Required	User UUID
userId	String	Required	User Id
userStoreId	String	Optional	Store ID if user is from external store
segmentId	String	Optional	When userId is not globally unique, segment must be specified.
rpId	String	Required	An identifier for the Relying Party.
statusAfterAssignment	String	Optional	Token status to set after token is assigned.

REST Sample Code

FIDO2 Token Pre-Reg Assign Credential API Call	
URL: /FIDO2/pre-reg/assignCredential	
Request	
JSON	
<pre>{ "sessionToken": "string", "serialNumber": "1234567", "userId": "John", "rpId": "i-sprint.com" }</pre>	
Response	
JSON	

FIDO2 Token Pre-Reg Assign Credential API Call

```
{
  "result": {
    "message": "Token [1234567] successfully assigned to User [John].",
    "params": {
      "tokenStatus": "Activated",
      "tokenSerialNumber": "1234567",
      "userUUID": "",
      "tokenUUID": "",
      "rpId": "i-sprint.com",
      "userId": "John"
    }
  },
  "amProcessingTimeMillis": 166
}
```

FIDO2 Token Pre-Reg Unassign Credential

Use /FIDO2/pre-reg/unassignCredential to unassign a pre-registered token from a user. This API can also be used to remove user bindings stored within the token.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
tokenUUID	String	Required	Token UUID
serialNumber	String	Required	Serial number of token.
userUUID	String	Required	User UUID
userId	String	Required	User Id
userStoreId	String	Optional	Store ID if user is from external store
segmentId	String	Optional	When userId is not globally unique, segment must be specified.
removeAssignee	String	Optional	If true, the user binding stored in the token will be removed.
statusAfterUnassignment	String	Optional	Token status to set after token is unassigned.

REST Sample Code

FIDO2 Token Pre-Reg Unassign Credential API Call

URL: /FIDO2/pre-reg/unassignCredential

Request

JSON

```
{
  "sessionToken": "string",
  "serialNumber": "1234567",
  "userId": "John"
}
```

Response

JSON

```
  "result": {
    "message": "Token [1234567] successfully unassigned
from User [John].",
    "params": {
      "tokenStatus": "Unassigned",
      "tokenSerialNumber": "1234567",
      "tokenUUID": "7654321"
    }
  },
  "amProcessingTimeMillis": 786
}
```

5. UAS FIDO2 Token Configuration

Setup Steps

This section defines the steps relating to the configuration of the UAS FIDO2 Token solution.

- **Upload FIDO2 License**

A server license is used to enable general and product-specific features. Refer to [Upload FIDO2 License](#) for details.

- **Configure the FIDO2 Token Policy**

The FIDO2 token policy manages the behavior of the FIDO2 token(s) used for authentication. Refer to [Configure FIDO2 Token Policy](#) for policy details.

- **Configure the Token Batch**

Token batch refers to the physical grouping of token devices. This is normally used for hardware tokens. For example, this may be used to group tokens by batch of delivery from the token vendor. For software/mobile tokens, this is normally not used. However, all tokens, including software tokens, must belong to a token batch. Creation of the token batch can be done in the Admin Console or during token import using the Token Import Utility (TIU). Each batch is tied to a particular token policy. Refer to [Configure Token Batch](#) for configuration details.

- **Configure the Token Group**

Token group refers to the logical grouping of FIDO2 token devices. Token group is used for token management. For example, UAS contains a feature to manage system token assignment based on token groups. The creation of a token group can be done in the Admin Console or during token import using TIU. Refer to [Configure Token Group](#) for more details.

- **(Optional) Configure Lookup Module**

For usernameless authentication only. The FIDO2 discoverable credentials lookup module is a FIDO2 token-specific lookup module used for usernameless authentication. It must be used together with a second lookup module corresponding to the user's user store. Refer to [Configure Lookup Module](#) for configuration details.

- **Configure Login Module**

The FIDO2 token login module is a FIDO2 token-specific login module that provides an FIDO2 token-based login method for users who may reside in either an internal or external user store. Refer to [Configure Login Module](#) for the configuration steps.

- **Configure Authentication Realm**

Authentication realms associate user credentials with authentication method(s) based on a lookup module and login module(s). User entries can be created in a local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). Refer to [Configure Authentication Realm](#) for configuration details.

- **Assign Login Accounts to the User**

A user is an entity that accesses the system and may belong to a default store (local repository) or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to the user by adding it to the **Login Accounts** tab. Refer to [Configure User Accounts](#) for details.

- **Configure Admin Rights**

Authorization is required to use the REST API Token Service for UAS FIDO2 token operations. Refer to [Configure Admin Rights](#) for the list of permissions that need to be granted.



Important Note: Instead of performing the configurations below, the UAS FIDO2 token solution can also be used out-of-the-box with no additional configurations. A default FIDO2 authentication realm (DefaultFIDO2Realm) is created in AccessMatrix upon the initial startup. It is already configured with a pre-created:

- Lookup module (DefaultFIDO2MultiChoiceLookup)
- Lookup module (DefaultFIDO2DCLookup)
- Login module (DefaultFIDO2TokenLogin)
- Token policy (DefaultFIDO2TokenPolicy)
- Token batch (DefaultFIDO2TokenBatch)
- Token group (DefaultFIDO2TokenGroup)

If you wish to use FIDO2 without further configurations, simply skip to the [Configure User Accounts](#) section and assign the default FIDO2 realm to a user to get started.

If you wish to create your own FIDO2 authentication realm, or learn more about the configurable attributes for each of the above, you can proceed to view all of the steps below.

Upload FIDO2 License

Before performing any configurations in the Admin Console, secure the server license required to implement FIDO2. Ensure that the FIDO2 vendor is added in the **Token Vendors** license feature to enable FIDO2 support.

To load the server license:

-
1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
 2. Select the **Server Id** and click **Edit**.
 3. Select the **Licensed Features** tab and select the first radio button. Browse for the license file.
 4. Click **Upload Licence File** to upload the license file to the server.
 5. Save the changes.
 6. Restart the AccessMatrix server for the new license to take effect.

Configure FIDO2 Token Policy

To configure the FIDO2 token policy, navigate to **Administration Dashboard > Security Policy > Token** and click **Token Policy** on the left pane. Click **Create**, select the "FIDO2 Token Policy" implementation from the **Choose Token Policy Implementation** page and click **Next** button. Then, specify the details for the token policy attributes before saving the changes.

Attribute Name	Description
Attributes Tab	
Encryption Key	Token encryption key used to encrypt the token blob.
Lock Token If Bad Verification Count Is More Than	Specifies the number of bad verification attempts to be exceeded before locking the token. If specified as "0", any number of bad verification attempts will not lock the token. Range of values: 0 to 100 Default: 3
Relying Party Whitelist	If specified, only the configured Relying Party/Parties will be allowed to be used for registration with this policy. If left unspecified, all Relying Parties will be allowed.
Maximum Allowable Number Of Credentials Registered Per RP	Limits the number of credentials a user can register on a Relying Party. Value should not be less than 1. Default: 1
FIDO2 Registration	

Attribute Name	Description
User Verification	If specified, UAS will enforce this registration option from the Relying Party. User verification refers to the process of a user verifying their identity with an authenticator. For example, user verification can be done using biometrics on a mobile phone, or by using a PIN on a laptop. Values: • "Discouraged" User verification will not be used. However, if the authenticator only supports user verification, it will still be allowed to use user verification. • "Preferred" User verification will be used where possible. However, if the authenticator does not support user verification, it will still be allowed to skip user verification. • "Required" User verification must be used. Default: Not Specified
Discoverable Credentials	If specified, UAS will enforce this registration option from the Relying Party. Discoverable credentials refer to FIDO2 credentials that allow for authentication without requiring a username to be entered (i.e. usernameless authentication). Values: • "Discouraged" The FIDO2 credential to be created should not allow usernameless authentication. However, if the authenticator only supports usernameless authentication, it will still be allowed to use usernameless authentication. • "Preferred" The FIDO2 credential to be created should support usernameless authentication where possible. However, if the authenticator does not support this, a non-discoverable credential will be created instead. Non-discoverable credentials will require users to provide a username during authentication. • "Required" The FIDO2 credential to be created must support usernameless authentication. Default: Not Specified

Attribute Name	Description
Authenticator Attachment	If specified, UAS will enforce this registration option from the Relying Party. Authenticator attachment refers to how an authenticator is tied with a client device. Values: • "Cross-Platform" The authenticator can be used across multiple devices. For example, a USB hardware token can be plugged into different computers. • "Platform" The authenticator is built into a device and cannot be used across multiple devices. For example, a Trusted Platform Module (TPM) that is built into a laptop and cannot be used on a different laptop. Default: Not Specified
Attestation	If specified, UAS will enforce this registration option from the Relying Party. Attestation refers to the process of verifying the origin of an authenticator. As part of this process, the authenticator sends an attestation statement that includes verifiable information such as its manufacturer and model. Values: • "None" The authenticator is not required to provide information about its origin during registration. • "Direct" The authenticator is required to provide information about its origin during registration. Default: Not Specified
FIDO2 Assertion	
User Verification	If specified, UAS will enforce this assertion option from the Relying Party. User verification refers to the process of a user verifying their identity with an authenticator. For example, user verification can be done using biometrics on a mobile phone, or by using a PIN on a laptop. Values: • "Discouraged" User verification will not be used. However, if the authenticator only supports user verification, it will still be allowed to use user verification. • "Preferred" User verification will be used where possible. However, if the

Attribute Name	Description
	authenticator does not support user verification, it will still be allowed to skip user verification. "Required" User verification must be used. Default: Not Specified
Restriction Policy	This configuration allows AccessMatrix to enforce a whitelist or blacklist on the assertion process, by using AAGUID information. Default: Not Specified Note: AAGUID Blacklist/Whitelist does not support UAF/U2F tokens as they lack AAGUID information.
Restriction Policy AAGUID List	Specifies a list of AAGUIDs that the Restriction Policy will be applied to. Note: The AAGUID is represented as a string (e.g. "7a98c250-6808-11cf-b73b-00aa00b677a7") consisting of 5 hex strings, each separated by a dash ("-").
FIDO2 Attestation Configuration	
Allow Self/None Attestation	Indicates if the server is to accept authenticators using 'self' or 'none' attestation types. If "false", the server rejects authenticators which have defined their attestation type as 'self' or 'none'. These authenticators will not be able to complete the authenticator registration process. Default: true
FIDO2 Metadata JWT	Specifies the file paths to the FIDO2 JWT metadata blob and certificate respectively. Each FIDO2 JWT metadata blob will be verified using the corresponding certificate provided. Both of the files must be provided together if the RP requires attestation. Multiple file pairs are supported.
FIDO2 Metadata JSON	Specifies the file path to the JSON FIDO2 metadata statement. This file contains verifiable information about a specific authenticator. Multiple files are supported.

For example:

1. Navigate to **Administration Dashboard > Security Policy > Token** and click **Token Policy** on the left pane.
2. Click **Create**, select the "FIDO2 Token Policy" implementation from the **Choose Token Policy Implementation** page, and click **Next** button.

-
3. Enter a **Token Policy Id** (e.g. "FIDO2Policy") and a **Token Policy Name** (e.g. "FIDO2 Policy").
 4. Under the **Attributes** tab, provide values for the following attributes:
 - **Relying Party Whitelist** - Specify the Id(s) of the Relying Party/Parties that are allowed to be used for FIDO2 token registration (e.g. "fido2.i-sprint"). Click **Add**. Relying Party refers to the server requesting access to online resources.
If left blank, all Relying Parties are allowed.
 - **Maximum allowable number of credentials registered per RP** - Specify the maximum number of credentials a user is allowed to register for an RP (e.g. "1").
The value should not be less than 1.
 5. Under the **FIDO2 Attestation Configuration** tab, provide values for the following attributes:

i **Note:**

- Depending on the authenticator's vendor, either the FIDO2 JWT metadata blob and certificate or JSON FIDO2 metadata statement file will be provided.
- The FIDO2 JWT metadata blob and certificate work in conjunction and must be used together. The certificate is provided alongside the FIDO2 JWT metadata blob as a pair.

- **Allow Self/None Attestation** - Using the drop-down list, select **(default)**.
- **FIDO2 Metadata Files** - Specify the file paths for the FIDO2 JWT metadata blob and certificate respectively, if these files have been provided by the authenticator's vendor. Click **Add**. If you have multiple file pairs, you may repeat this process as needed. Each FIDO2 JWT metadata blob will be verified using the corresponding certificate provided. The FIDO2 JWT metadata blob and certificate can be obtained from the authenticator's vendor. For example, the FIDO Alliance [provides a certificate](#) along with their JWT metadata blob.
-

i **Note:** If the FIDO Alliance metadata blob and certificate are used, the administrator should be aware of the following:

- The metadata blob itself contains two certificates with their own expiry dates. The administrator must take note of each expiry date. They must replace the current blob file with the latest blob file from the FIDO Alliance when either certificate is about to expire (and update the **Blob file path**, if necessary).
- The external certificate used to verify the blob also has its own separate expiry date. The administrator must take note of this

expiry date. They must replace the current certificate with the latest version from the FIDO Alliance when it is about to expire (and update the **Certificate** file path, if necessary).

- **JSON Metadata Statement File Path** - Specify the file path to the JSON FIDO2 metadata statement, if this file has been provided by the authenticator's vendor. Click **Add**. If you have multiple files, you may repeat this process as needed. The JSON FIDO2 metadata statement can be obtained from the authenticator's vendor. This is the file that contains verifiable information about a specific authenticator.

6. Click **Save**.

Configure Token Batch

To configure a token batch, navigate to **Operation Dashboard > Token > Token Batch** and click the **Create** button. Provide the details for the token batch attributes before saving the changes.

Attribute Name	Description
*Token Batch Id	Identification assigned to the token batch.
*Token Batch Name	The name of the token batch.
Description	Brief description for the token batch.
Attributes Tab	
*Token Policy	Specify the FIDO2 token policy, e.g. "FIDO2Policy".
Static Vector	(Not applicable)
Manufacture Time	The date/time when the token is manufactured.
Expiry Time	The token expiry date/time.
Token Attributes Template Id	The identification of the template is used to set token attributes for additional custom information.

For example:

1. Navigate to **Operation Dashboard > Token > Token Batch** and click **Create**.
 2. Enter a **Token Batch Id** (e.g. "FIDO2Batch") and a **Token Batch Name** (e.g. "FIDO2 Batch").
 3. Under **Token Policy**, specify the Id of the previously-created policy (e.g. "FIDO2Policy").
-

4. Provide details for the remaining token batch attributes, if necessary.
5. Click **Save**.

i **Note:** For more details on the remaining token batch attributes, refer to [UAS Administration Guide - Token Batch](#).

Configure Token Group

To configure a token group, navigate to **Operation Dashboard > Token > Token Group** and click the **Create** button. Provide the details for the token group attributes before saving the changes.

Attribute Name	Description
*Token Group Id	Identification assigned to the token group.
*Token Group Name	The name of the token group.
Description	Brief description of the token group.
*Token Vendor	The token vendor to manage in this group. Ensure that the correct token vendor (i.e. FIDO2) is configured to group similar tokens.
Segment	Name of the segment the token group belongs to.
Attributes Tab	
System Token Assignment	
By default, the system will assign the oldest token first, based on the time the token is placed in a token group. This is particularly useful for hardware tokens that have an expiry date due to the lifespan of the battery. For soft tokens, this is no longer required. Software tokens have a higher rate of token assignment as users change their devices (such as mobile phones, etc.) more often than hardware tokens. A soft token vendor such as V-Key sometimes requires the new tokens to be re-provisioned when they update their firmware. To allow for a higher system-token assignment capacity and to increase performance, it is highly recommended that you set Randomize Reservation Pool to "True" and Database Fetching Order of Reservation Pool to "None". These settings fetch the token reservation pool from the database quicker and reduce the chance of conflicts and clashes during the token assignment for a more efficient system-token-assignment process. If the system expects a more extensive concurrent system-token-assignment request, increase the value of Reservation Pool Size.	
Randomize Reservation Pool	This is to randomize the available tokens when assigning to a user if there are large system token assignment concurrent requests. Select the setting "true" for soft tokens that may require token re-provisioning due to soft token SDK/firmware upgrade. It will fetch the token reservation pool from the database much quicker and reduce the chance of conflicts and clashes during

Attribute Name	Description
	the token assignment. Default: false
Reservation Pool Size	Set the reservation pool size for random system token assignment. When handling high concurrent requests, set Randomize Reservation Pool to "true" and then set this pool size to at least 20. Default: 10
Database Fetching Order of Reservation Pool	This setting specifies the sorting of fetch from the database to get the reservation pool. Select the setting "None" for soft tokens that may require token re-provisioning due to soft token SDL/firmware upgrades, such as V-Key soft Tokens. Setting the order to "None" will reduce the database processing tremendously. Default: With Ordering

For example:

1. Navigate to **Operation Dashboard > Token > Token Group** and click **Create**.
2. Enter a **Token Group Id** (e.g. "FIDO2Group") and a **Token Group Name** (e.g. "FIDO2 Group").
3. Provide details for the remaining token batch attributes, if necessary.
4. Click **Save**.



Note: For more details on the remaining token group attributes, refer to [UAS Administration Guide - Token Group](#).

Configure Lookup Module

The AccessMatrix UAS system supports usernameless authentication for FIDO2, wherein the user is not required to provide a username in the authentication process. This feature is also known as ID-less authentication. To enable this feature, you are required to configure the FIDO2 discoverable credentials lookup module. The instructions are detailed below:

Configure FIDO2 Discoverable Credentials Lookup Module

Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module** and create a new lookup module. Select the "FIDO2 Discoverable Credentials Lookup" implementation and click the **Next** button. Then, specify the details for the lookup module attributes before saving the changes.

Attribute Name	Description
*User Lookup Module Id	Unique Id of user lookup module.
*User Lookup Module Name	Name of user lookup module.
Description	Description of user lookup module.
Implementation	Type of user lookup module implementation.

For example:

1. Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module** and click **Create**.
2. Select the "FIDO2 Discoverable Credentials Lookup" implementation on the **Choose User Lookup Module Implementation** page and click **Next**.
3. Enter a **User Lookup Module Id** (e.g. "FIDO2DCLookup") and a **User Lookup Module Name** (e.g. "FIDO2 Discoverable Credentials Lookup").
4. Click **Save**.



Note: You do not need to configure the user store for this lookup module.

Configure Multiple Choice Lookup Module

The multiple choice lookup module functions as a mechanism to link lookup modules so they may be used together. To enable usernameless authentication for FIDO2, the multiple choice lookup module must be used to link the FIDO2 discoverable credentials lookup module to a second lookup module. The second lookup module should be the lookup module corresponding to the user store in which the user is located (e.g. "SystemLookup"). Ensure that the lookup modules have been created before proceeding.



Note: You can refer to [AccessMatrix Common Administration Guide - Configure User Lookup Modules](#) for information on the lookup modules that may be relevant.

Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module** and create a new lookup module. Select the "Multiple Choice Lookup" implementation and click the **Next** button. Then, specify the details for the lookup module attributes before saving the changes.

Attribute Name	Description
*User Lookup Module Id	Unique Id of user lookup module.
*User Lookup Module Name	Name of user lookup module.
Description	Description of user lookup module.
Implementation	Type of user lookup module implementation.
Configuration Tab	
This tab consists of three lookup module attributes to support identity lookup chaining.	

For example:

1. Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module** and click **Create**.
2. Select the "Multiple Choice Lookup" implementation on the **Choose User Lookup Module Implementation** page and click **Next**.
3. Enter a **User Lookup Module Id** (e.g. "FIDO2MCLookup") and a **User Lookup Module Name** (e.g. "FIDO2 Multiple Choice Lookup").
4. Under the **Configuration** tab, for the attribute **First Lookup Module**, select the lookup module that was previously configured (e.g. "FIDO2DCLookup").

Note: For better performance, the first lookup module should be the FIDO2 discoverable credentials lookup.

5. For the attribute **Second Lookup Module**, select the secondary lookup module corresponding to the user store the user is located in (e.g. "SystemLookup").
6. Click **Save**.

Configure Login Module

Navigate to **Administration Dashboard > Configuration > Authentication > Login Module** and create a new login module. Select the "FIDO2 Token Login" implementation. Click the **Next** button and provide the details for the login module attributes before saving the changes.

Attribute Name	Description
Login Configuration	
Login Policy	Refer to the description found in the System Password Basic attribute table.
Authentication Level	Refer to the description found in the System Password Basic attribute table. Default: 1
Relying Party Configuration	
Relying Party ID	The Relying Party server domain. It will automatically be added into the list of allowed origins.
Origins	The origin(s) of the WebAuthn request. Multiple values can be added. If not specified, the origin will be derived from the values specified for the Relying Party ID and Relying Party Port attributes.
FIDO2 Token Configuration	
Token Batch	The FIDO2 token batch Id.
Token Group	The FIDO2 token group Id.

For example:

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
2. Click **Create** and select the "FIDO2 Token Login" implementation from the **Choose Login Module Implementation** page:
3. Enter a **Login Module Id** (e.g. "FIDO2LM") and a **Login Module Name** (e.g. "FIDO2 Login Module").
4. Configure the following attributes under the **Relying Party Configuration** header:
 - **Relying Party ID** - Specify the Relying Party server domain (e.g. "fido2.i-sprint.com").
 - **Relying Party Port** - Specify the port of the Relying Party server (e.g. "8443").
 - **Origins** - Specify the origin(s) to be used as the WebAuthn origin(s). If unspecified, the origin will be derived from the RP ID and port configured in the previous two attributes. You may need to specify the origin(s) in situations where the FIDO2 requester uses a non-traditional origin format that does not match the address derived from the RP ID and port. For example, if a mobile native app makes a token registration request, the app key may be used as the origin instead of the RP address.
5. Configure the following attributes under the **FIDO2 Token Configuration** header:

- **Token Batch** - The FIDO2 token batch Id (e.g. "FIDO2Batch").



Note: This should match the Id of the token batch configured in the respective section.

- **Token Group** - The FIDO2 token group Id (e.g. "FIDO2Group").



Note: This should match the Id of the token group configured in the respective section.

6. Click **Save**.

Configure Authentication Realm

Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm** and create a new authentication realm. Provide the details for the authentication realm attributes before saving the changes.

Attribute Name	Description
*Authentication Realm Id	Unique Id of authentication realm.
*Authentication Realm Name	Name of authentication realm.
Description	Description of authentication realm.
Version	Authentication realm entry version.
Configuration Tab	
Automatically assigned to users	Allow this authentication realm to be automatically assigned to all users.
User Lookup Module	Lookup module is used to validate user identity (i.e. user search in the user store(s) or other validation methods).
First(1st) Login Module	The first login method to use for user authentication.
Second(2nd) Login Module	Optional subsequent user authentication method.
Third(3rd) Login Module	Optional subsequent user authentication method required.
Fourth(4th) Login Module	Optional subsequent user authentication method required.

Attribute Name	Description
Fifth(5th) Login Module	Optional subsequent user authentication method required.
Enable Login/Logout History Saving	Set this to false to increase performance if tracking of login/logout history is not required in your project or during a stress test. Default: true
User Response	On successful authentication, return name-value pair responses specified in this User Response.
Realm-Based Session Policy	The realm-based session policy of this authentication realm. If no realm-based session policy is assigned, then the session policy defined in Global Setting will be used.

For example:

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
2. Click **Create**.
3. Enter an **Authentication Realm Id** (e.g. "FIDO2Realm") and an **Authentication Realm Name** (e.g. "FIDO2 Realm").
4. Specify the **User Lookup Module** value that will be used to validate user identity (e.g. "SystemLookup").



Note: To allow flexible FIDO authentications (both ID and ID-less authentication), use the multiple choice lookup module configured in the earlier step (e.g. FIDO2MCLookup). This lookup module should contain a FIDO2 discoverable credentials lookup module.

5. Specify the newly-created **First(1st) Login Module** that will be used for user authentication (e.g. "FIDO2LM").
6. Click **Save**. A new authentication realm is created.
7. Assign this realm to a user.

Configure User Accounts

A user may belong to a default store or external user store. Refer to [AccessMatrix Common Administration Guide - Manage User Stores](#) to know about the different types of user stores and how to configure it.

To create a new user in the default store, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Provide values to the following attributes that are necessary for the FIDO2 token implementation before saving the changes.

Attribute Name	Description
*User Id	Unique user Id.
*User Name	Name of user.
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of User Store the user belongs to, i.e. "Default Store".
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to the user. Click Add to display the Choose Authentication Realm dialog box. Select the authentication realm created in the previous section and click OK . Both the authentication realm and the login module(s) associated with it will be assigned to the user.	

i **Note:** For more details about configuring user accounts, refer to [AccessMatrix Common Administration Guide - Manage Users](#)

Configure Admin Rights

To use the REST API Token Service for UAS FIDO2 token operations, specific authorizations are required for each API call. As such, the non-interactive user sending the API call must be configured with the necessary admin rights. The admin rights required are listed in the table below.

i **Note:** This assumes that the non-interactive user will be performing all UAS FIDO2 token operations. For more details on the specific permissions required by each API (e.g. token registration), please refer to [Token Service](#) and its respective pages.

Resource	Operation
User Admin Rights	
User	Assign Token
	View

Resource	Operation
Token Management Rights	
Token	Create
	Delete
	Edit
	View

Ensure that the user's admin role has the necessary admin rights.

For example:

1. Navigate to **Administration Dashboard > Security Policy > Admin & Audit > Admin Role**.
2. Click on the relevant admin role under **Admin Role Id** that will be performing the FIDO2 token operations.
3. Verify that the permissions listed in the table above has a tick in their respective checkboxes. If not, proceed to the next step.
4. Click **Edit**.
5. Click on the relevant permissions which have their checkboxes unmarked.
6. Click **Save**.



Note: For more details on how to create, configure an admin role and assigning it to users, you may refer to [Using AccessMatrix REST APIs - Configuring Admin Rights for Using AccessMatrix REST APIs](#).

6. UAS FIDO2 Token Implementation Using API

AccessMatrix Universal Authentication Server (UAS) provides various APIs for implementing the FIDO2 tokens. These endpoints are broadly categorized into token operations and authentication use cases:

- [Token Service](#)
- [Authentication Service](#)

Pre-requisites

Before attempting any of the above, please ensure that the following pre-requisites are met:

- The relevant user has already been created in the AccessMatrix server.
- The token batch and token group have already been created and configured.
- If authenticator attestation is enabled in the FIDO2 token policy used, the following must be provided:
 - Certificate of the metadata blob
 - Metadata blob in JWT format
- The non-interactive user has already been created and configured with the necessary admin rights to send these API calls.



Note: You may refer to the [UAS FIDO2 Token Configuration](#) page for details on all of the above pre-requisites.

- The necessary web client and Relying Party server implementation steps have been completed. These steps are project-specific and are not covered within the scope of this document.

6.1. Token Service

The REST API Token Service is used for UAS FIDO2 token operations such as token lifecycle management, as well as token verification. In a typical use case, these APIs allow for easy implementation of FIDO2 tokens as part of two-factor authentication (2FA) processes.

The following sections provide detailed information on performing the various FIDO2 token operations:

- [Token Registration](#)
- [Token Assertion](#)
- [Token Verification](#)
- [Token Management](#)

6.1.1. Token Registration

This section covers the APIs used for token registration, which allows for FIDO2 tokens to be registered to users. This operation is supported by the REST API Token Service URL /FIDO2.

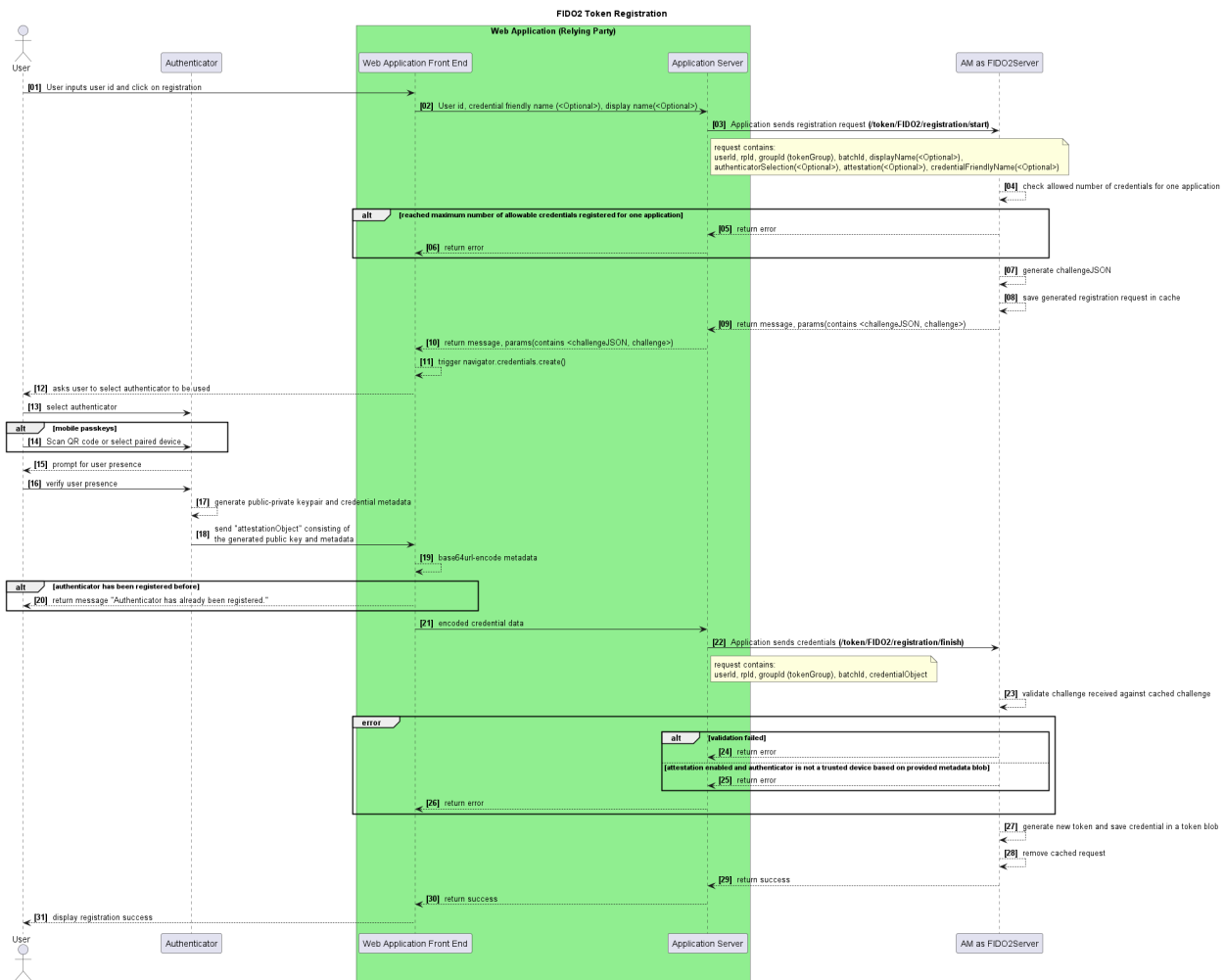
Permissions Required

The following admin rights are required to send this API call:

Resource	Operation
User Admin Rights	
User	Assign Token
	View
Token Management Rights	
Token	Create
	Edit
	View

Sequence Diagram

The sequence diagram below shows the overall flow between the User, Authenticator, Relying Party (the Web Application Front End and its Application Server), and AccessMatrix (AM) as the FIDO2 Server. The Relying Party implementation is project-specific and will not be covered in this scope.



1. The user provides their `userId` to the RP.
2. The web application calls the `/FIDO2/registration/start` API.
3. The FIDO2 Server returns a set of parameters containing all the metadata needed to generate a credential, as well as a challenge.
4. The web application uses the parameters to generate a new credential on the authenticator.
5. The web application calls the `/FIDO2/registration/finish` API with the generated credential information and the challenge.
6. The FIDO2 Server validates the challenge. If the validation is successful, it stores the new credential in the database.

FIDO2 Token Registration Start

Use `/FIDO2/registration/start` to start the user registration process and obtain a set of parameters that will be used to generate a credential.

The response will contain a challengeJSON JSON string containing the various parameters generated by the FIDO2 server, as well as a challenge string that has been extracted from challengeJSON. The parameters in challengeJSON will be used by the client to generate a credential for the authenticator.



Note: The challengeJSON contains user data that is returned by the AccessMatrix server. The following illustrates how the credential is generated based on the user information returned within the challengeJSON.

- user.name: <storeId>:<segmentId>:<userId>
- user.id: <userUUID>
- user.displayName: <displayName, if specified, else userId>

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
userId	String	Required	User's ID.
userStoreId	String	Optional	Store ID if user is from external store. If unspecified, AM will search for users in the default user store.
segmentId	String	Optional	When user ID is not globally unique, segment must be specified.
batchId	String	Required	ID of token batch.
groupId	String	Required	ID of token group.
rpld	String	Required	An identifier for the Relying Party.
origin	String	Optional	Absolute URL of the Relying Party or an APK hash. An absolute URL refers to the full URL of the web application, e.g. "https://webapp.com". This parameter can be used to specify a custom port. Note: If not provided, the origin will be resolved to the rpld with port 443.
displayName	String	Optional	User-defined name for the user to be used for display purposes.
attestation	String	Optional	Must be a value of 'direct', 'indirect' or 'none'. If not provided, value 'none' will be used.

Parameter		Data Type	Required/ Optional	Description
credentialFriendlyName		String	Optional	A user defined name to identify the credential. If not provided, will use 'FIDO2Credential'.
authenticatorSelection		Object	Optional	
	userVerification	String	Optional	Must be a value of 'preferred', 'discouraged' or 'required'. If not provided, value 'preferred' will be used
	requireResidentKey	boolean	Optional	Either the requireResidentKey or residentKey fields can be specified. If both fields are specified, this field takes precedence. Default: false
	residentKey	String	Optional	Either the requireResidentKey or residentKey fields can be specified. If both fields are specified, the requireResidentKey field takes precedence. Must be a value of 'preferred', 'discouraged' or 'required'. Default: discouraged
	authenticatorAttachment	String	Optional	Must be a value of 'platform' or 'cross-platform'. If not provided, this field will not be used.

REST Sample Code

FIDO2 Token Registration Start API Call
URL: /FIDO2/registration/start

FIDO2 Token Registration Start API Call

Request

JSON

```
{
  "sessionToken": "string",
  "userId": "fido2user",
  "userStoreId": "ActiveDirectory",
  "segmentId": "SampleSegment1.1",
  "batchId": "fido2tokenbatch",
  "groupId": "fido2tokengroup",
  "rpId": "i-sprint.com",
  "displayName": "John",
  "authenticatorSelection": {
    "userVerification": "preferred",
    "requireResidentKey": "false",
    "authenticatorAttachment": "cross-platform"
  },
  "attestation": "direct",
  "credentialFriendlyName": "MyCredential"
}
```

Response

JSON

```
{
  "result": {
    "message": "Successful Preauthentication.",
    "params": {
      "challengeJSON": "",
      "challenge": ""
    }
  }
}
```

FIDO2 Token Registration Finish

Use /FIDO2/registration/finish to finish the registration process.

The response from the client is validated using the challenge that was generated and stored after /FIDO2/registration/start was called. Once the response has been successfully validated, a FIDO2 token will be generated based on the credentials provided and stored in the server.

Parameter	Data Type	Required/ Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
userId	String	Required	User's ID.
userStoreId	String	Optional	Store ID if user is from external store. If unspecified, AM will search for users in the default user store.
segmentId	String	Optional	When user ID is not globally unique, segment must be specified.
credentialObject	String	Required	Result of the successfully resolved promise, containing Base64 URL encoded attributes from the client side in JSON string format to be returned to FIDO2 server to be verified.
batchId	String	Required	ID of token batch.
groupId	String	Required	ID of token group.
rpId	String	Required	An identifier for the Relying Party.
origin	String	Optional	Absolute URL of the Relying Party or an APK hash. absolute URL refers to the full URL of the web application, e.g. "https://www.webapp.com". This parameter can be used to specify a custom port. Note: If not specified, the origin will be resolved to the rpId with port 443.
statusAfterRegistration	String	Optional	Token status to be set after successful registration. Note: You may refer to the list of accepted values in Token States .
template	String	Optional	The template parameter is a string of template numbers separated by comma, e.g. "1,2,3" that specifies which fields are to be returned. Individual field names can be passed to make the template more specific. Field names can also be combined with numbers, e.g. "1,batch,assignedUserIds". Note: You may refer the list of templates and field names in Templates .

REST Sample Code

FIDO2 Token Registration Finish API Call

URL: /FIDO2/registration/finish

Request

JSON

```
{
  "sessionToken": "string",
  "userId": "fido2user",
  "userStoreId": "ActiveDirectory",
  "segmentId": "SampleSegment1.1",
  "credentialObject": "string",
  "batchId": "fido2tokenbatch",
  "groupId": "fido2tokengroup",
  "rpId": "i-sprint.com",
  "statusAfterRegistration": "Activated"
}
```

Response

JSON

```
{
  "result": {
    "serialNum": "0WATDuN7HyUkpZlY3V4LgefFTJ7yfZXw",
    "vendor": "FIDO2",
    "model": "FIDO2",
    "UUID": "nlyloYSvpJXRBeMAfj7Pt89gnb1LGliSKCnF4ZKyAx8",
    "id": "FIDO2-0WATDuN7HyUkpZlY3V4LgefFTJ7yfZXw",
    "status": "Unassigned",
    "version": 0
  },
  "amProcessingTimeMillis": 531
}
```

6.1.2. Token Assertion

This section covers the APIs used for FIDO2 token assertion, which facilitates the authentication process using FIDO2 tokens. This operation is supported by the REST API Token Service URL /FIDO2.



Note: FIDO2 token assertion is conceptually similar to authentication for non-FIDO2 tokens, such as OATH tokens.

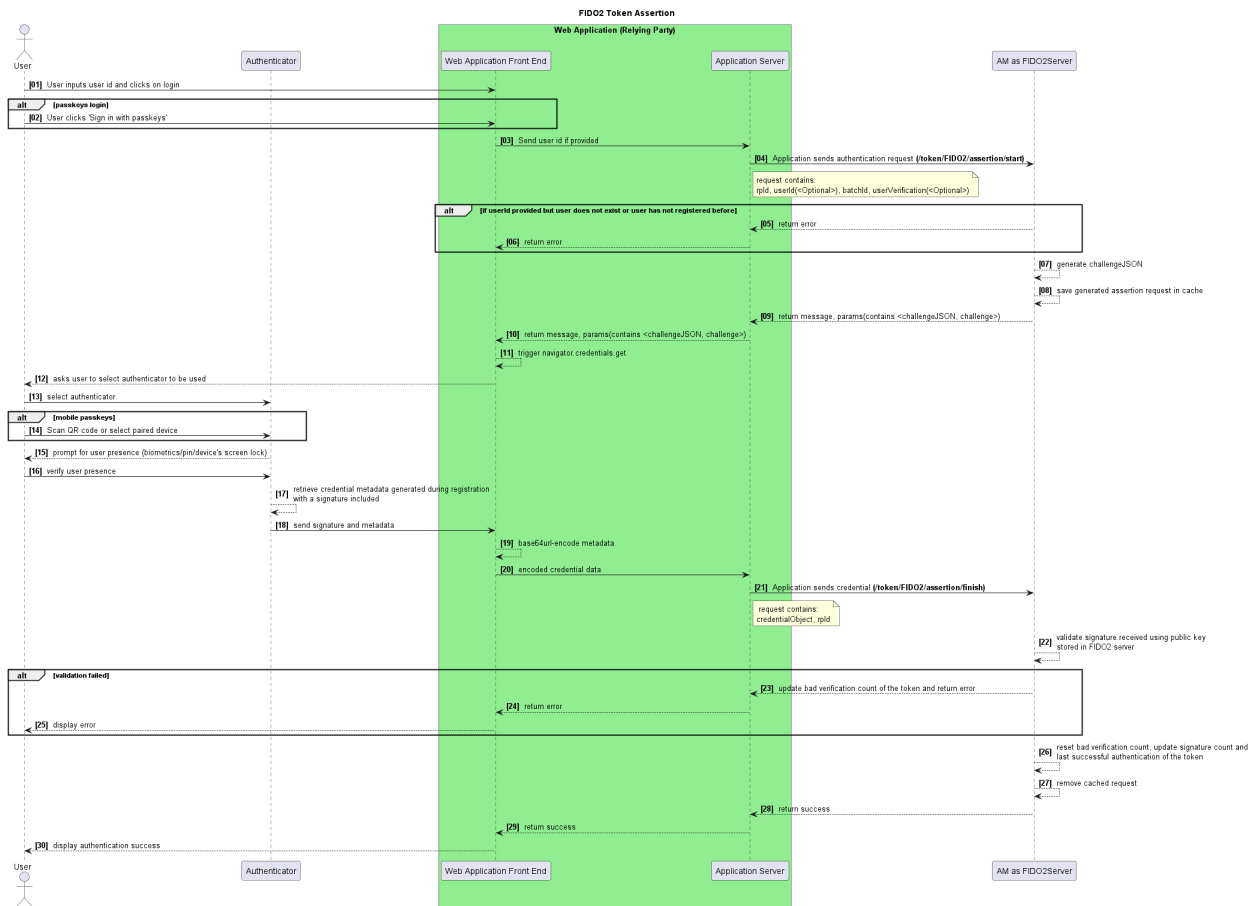
Permissions Required

The following admin rights are required to send this API call:

Resource	Operation
User Admin Rights	
User	View
Token Management Rights	
Token	Edit
	View

Sequence Diagram

The sequence diagram below shows the overall flow between the User, Authenticator, Relying Party (the Web Application Front End and its Application Server), and AccessMatrix (AM) as the FIDO2 Server. The Relying Party implementation is project-specific and will not be covered in this scope.



1. (Optional) The user provides their `userId` to the RP.
2. The web application calls the `/FIDO2/assertion/start` API.
3. The FIDO2 Server returns a challenge with a set of allowed credentials to be used for authentication. If `userId` was not provided in step 2, the FIDO2 Server will not return the set of allowed credentials.
4. The web application uses the set of credentials to retrieve the correct credential to be used for authentication from the authenticator. If `userId` was not provided in step 2, it means that `requireResidentKey` was enabled during the registration process and a private key that is associated with a user handle is generated on the authenticator. Based on the origin, the user will have to select the matching credential returned from the authenticator for that particular RP to be used for the assertion process.
5. The web application calls the `/FIDO2/assertion/finish` API to validate the assertion signature.
6. The FIDO2 Server validates the assertion signature. If the validation is successful, it resets the bad verification count, updates the signature counter and the last successful authentication in

the FIDO2 token and saves the token in the database. If the validation fails, the bad verification count is incremented and stored in the FIDO2 token before being saved into the database.

FIDO2 Token Assertion Start

Use `/FIDO2/assertion/start` to start the authentication process and generate the necessary authentication parameters.

The generated authentication parameters are returned to the RP, where they are used to retrieve credentials from the authenticator.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
userId	String	Optional	User's ID.
userStoreId	String	Optional	Store ID if user is from external store. If unspecified, AM will search for users in the default user store.
segmentId	String	Optional	When user ID is not globally unique, segment must be specified.
batchId	String	Required	ID of token batch.
rpId	String	Required	An identifier for the Relying Party.
origin	String	Optional	Absolute URL of the Relying Party or an APK hash. An absolute URL refers to the full URL of the web application, e.g. "https://webapp.com". This parameter can be used to specify a custom port. Note: If not specified, the origin will be resolved to the rpId with port 443.
userVerification	String	Optional	Must be a value of 'preferred', 'discouraged' or 'required'. If not provided, the value 'preferred' will be used instead.
txnData	String	Optional	If txnData is not specified, <original_random_challenge> is returned. If txnData is specified, the challenge returned in the response will contain additional values concatenated with <original_random_challenge> in the

Parameter	Data Type	Required/Optional	Description
			following format: <timestamp>,<original_random_challenge>,<transaction_information> Note: The challenge will be Base64URL encoded.

REST Sample Code

FIDO2 Token Assertion Start API Call	
URL: /FIDO2/assertion/start	
Request	
JSON	
<pre>{ "sessionToken": "string", "userId": "fido2user", "userStoreId": "ActiveDirectory", "segmentId": "SampleSegment1.1", "rpId": "i-sprint.com", "userVerification": "discouraged", "batchId": "fido2TokenBatch", "txnData": "Transfer \$xxx to xxx." }</pre>	
Response	
JSON	
<pre>{ "result": { "message": "Successful Preauthentication.", "params": { "challengeJSON": "", "challenge": "" } } }</pre>	

FIDO2 Token Assertion Finish

Use /FIDO2/assertion/finish to finish the authentication process.

A signature is returned as a response from the RP. This signature is verified using credentials within the FIDO2 token that is stored in the database.

If the verification is successful, the signature counter in the FIDO2 token will be updated and saved in the database.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
credentialObject	String	Required	Result of the successfully resolved promise, containing Base64 URL encoded attributes from the client side in JSON string format to be returned to FIDO2 server to be verified.
rpId	String	Required	An identifier for the Relying Party.
origin	String	Optional	Absolute URL of the Relying Part of an APK hash. An absolute URL refers to the full URL of the web application, e.g. "https://webapp.com". This parameter can be used to specify a custom port. Note: If not provided, the origin will be resolved to the rpId with port 443.

REST Sample Code

FIDO2 Token Assertion Finish API Call	
URL: /FIDO2/assertion/finish	
Request	
JSON	
<pre>{ "sessionToken": "string", "credentialObject": "", "rpId": "i-sprint.com" }</pre>	
Response	
JSON	

FIDO2 Token Assertion Finish API Call

```
{
  "result": {
    "params": {
      "result": "success",
      "tokenSerialNumber": "798427653560",
      "credentialId":
"dLj1Lq3SU84hJNCyCpOVhKkgdqFA25yHuv_KnxMsHnrz0f2d0hjyvJRN16jhJepYDiBuNtYlm11
VsJaKVRzHLg",
      "userUUID": "WjZW7Kcumt",
      "tokenUUID": "mOwjSjuKMb",
      "userId": "demouser",
      "credentialFriendlyName": "credential0"
    }
  },
  "amProcessingTimeMillis": 110
}
```

6.1.3. Token Verification

This section covers the APIs used for the token verification of FIDO2 tokens. This operation is supported by the REST API Token Service URL `/token`, which provides a unified endpoint for performing token verification and allows developers to use a single endpoint across both FIDO2 tokens and all other tokens (e.g. OATH, SMS OTP, etc).



Note: The following APIs are similar to `/FIDO2/assertion/start` and `/FIDO2/assertion/finish`.

Generate Challenge

Use `/token/generateChallenge` to generate a FIDO2 challenge.

Parameter		Data Type	Required/Optional	Description
sessionToken		String	Required	User's session token or dynamic agent session.
UUID		String	Required	UUID of token.
serialNumber		String	Required	Serial number of token.
vendor		String	Required	Token vendor.
params		Object	Optional	
	userId	String	Required	User Id.
	userSegmentId	String	Optional	ID of segment that contains the user especially if userId is not unique globally.
	userStoreId	String	Optional	Store ID if user is from external store.
	rpId	String	Required	An identifier for the Relying Party.

Parameter		Data Type	Required/Optional	Description
	origin	String	Optional	Absolute URL of the Relying Party or an APK hash. An absolute URL refers to the full URL of the web application, e.g. "https://webapp.com". This parameter can be used to specify a custom port. Note: If not specified, the origin will be resolved to the rpId with port 443.
	userVerification	String	Optional	Must be a value of 'preferred', 'discouraged' or 'required'. If not provided, value 'preferred' will be used.
	txnData	String	Optional	Transaction information entered by end-user. Note: The maximum length is 2048 characters.

REST Sample Code

Generate Challenge API Call	
URL: /token/generateChallenge	
Request	
JSON	
<pre>{ "sessionToken": "string", "serialNumber": "92iCOTaD6Cw0GOB5zqlalyQsbNuI1VW4", "vendor": "FIDO2", "params": { "rpId": "i-sprint.com", "userId": "demouser" } }</pre>	
Response	
JSON	
<pre>{ "result": { "authenticationCode": -1, "message": "Successful preauthentication.", "params": { "tokenSerNum": "92iCOTaD6Cw0GOB5zqlalyQsbNuI1VW4", "tokenStatus": "Activated", "challengeJSON": "{\"extensions\": {}, \"userVerification\": \"preferred\", \"challenge\": \"xVOfJ" } } }</pre>	

Generate Challenge API Call

```
-J55nNi8ay-r3-L-
_lJGSQjy3nCxlSjo8WKvdE\", \"rpId\": \"www.fido2.com\", \"allowCredentials\": [{\
\"id\": \"92iCOTaD6Cw0GOB5mib4VERuSJMwClkCh-
jhHHXONbCRKBUYNpoPjUhWIYkqgn9B2KSduSVpT2bMFAOQgGNBW0tUAjcyHZ43PY9nYKDx3HvzxI
lUzqlalyQsbNuI1VW4\", \"type\": \"public-key\"}]]\",
    \"challenge\": \"xVOfJ-J55nNi8ay-r3-L-_lJGSQjy3nCxlSjo8WKvdE\",
    \"vendorId\": \"FIDO2\",
    \"tokenUUID\": \"kqmuXCnu4ZLaTANeUxW7YxWe9H9_J7_EAdJf6Feyhxy\",
    \"tokenModel\": \"FIDO2\"
  },
  \"amProcessingTimeMillis\": 350
}
```

Verify Response

Use `/token/verifyResponse` to verify the response for a given challenge.

Parameter		Data Type	Required/Optional	Description
sessionToken		String	Required	User's session token or dynamic agent session.
UUID		String	Required	UUID of token.
serialNumber		String	Required	Serial number of token.
vendor		String	Required	Token vendor.
challenge		String	Required	The challenge.
response		String	Required	The response to be verified. For FIDO2, this would be the resolved promise returned from the WebAuthn API call.
params		Object	Optional	
	rpId	String	Required	An identifier for the Relying Party.
	origin	String	Optional	Absolute URL of the Relying Party or an APK hash. An absolute URL refers to the full URL of the web application, e.g. "https://webapp.com". This parameter can be used to

Parameter		Data Type	Required/ Optional	Description
				specify a custom port. Note: If not specified, the origin will be resolved to the rpld with port 443.

REST Sample Code

Verify Response API Call	
URL: /token/verifyResponse	
Request	
JSON	
<pre>{ "sessionToken": "", "serialNumber" : "92iCOTaD6Cw0GOB5zqlalyQsbNuI1VW4", "vendor" : "FIDO2", "challenge" : "xVOfJ-J55nNi8ay-r3-L-_lJGSQjy3nCxlSjo8WKvde", "response": "{ \"rawId\": \"92iCOTaD6Cw0GOB5mib4VERuSJMwClkCh-jhHHXONbCRKBUYNpoPjUhWlYkqgn9B2KSduSVpT2bMFAOQgGNBW0tUAjcyHZ43PY9nYKDx3HvzxI1UzqlalyQsbNuI1VW4\", \"id\": \"92iCOTaD6Cw0GOB5mib4VERuSJMwClkCh-jhHHXONbCRKBUYNpoPjUhWlYkqgn9B2KSduSVpT2bMFAOQgGNBW0tUAjcyHZ43PY9nYKDx3HvzxI1UzqlalyQsbNuI1VW4\", \"response\": { \"clientDataJSON\": \"eyJ0eXB1Ijoia2ViYXV0aG4uZ2V0Iiwia2hhbGxlbmdlIjoieDVoSnFQT1EzMFJpY3FUUVhVM1o4OVlsVk0xN0VLTUlhRk94bUFRVU9XVSIsIm9yaWdpbiI6Imh0dHBzOi8vbG9jYWxob3N0OjgwODYiLCJjcm9zc09yaWdpbiI6ZmFsc2V\", \"signature\": \"MEUCIA6kI9VIUpGLWyaJgaQXFzZPrRqxokiRQT7SvGw4TYwwAiEA16V68O_hJ6QUK27i288yIIRtZ3wGCPebk6v6oRfd0Ww\", \"authenticatorData\": \"SZYN5YgOjGh0NBcPZHgW4_krrmihjLHmVzzuoMdl2MFAAACfw\" }, \"type\": \"public-key\", \"extensions\": {} }\", \"params\":{ \"rpId\": \"i-sprint.com\" } }</pre>	
Response	
JSON	

Verify Response API Call

```
{
  "result": {
    "authenticationCode": -1,
    "params": {
      "result": "success",
      "tokenSerNum": "92iCOTaD6Cw0GOB5zqlalyQsbNuI1VW4",
      "tokenStatus": "Activated",
      "tokenSerialNumber": "92iCOTaD6Cw0GOB5zqlalyQsbNuI1VW4",
      "vendorId": "FIDO2",
      "credentialId": "92iCOTaD6Cw0GOB5mib4VERuSJMwClkCh-
jhHHXONbCRKBUYNpoPjUhWIYkqgn9B2KSduSVpT2bMFAOQgGNBW0tUAjcyHZ43PY9nYKDx3HvzxI
lUzqlalyQsbNuI1VW4",
      "userUUID": "WjZW7Kcunt",
      "tokenUUID": "kqmuXCNu4ZLaTANeUxW7YxWe9H9_J7_EAdJf6FeyhxY",
      "tokenModel": "FIDO2",
      "userId": "demouser",
      "credentialFriendlyName": "FIDO2Credential"
    }
  },
  "amProcessingTimeMillis": 51
}
```

6.1.4. Token Management

This section covers the APIs used for managing registered FIDO2 tokens. This operation is supported by the REST API Token Service URLs /FIDO2.

Permissions Required

The following admin rights are required to send this API call:

Resource	Operation
User Admin Rights	
User	View
Token Management Rights	
Token	View
	Edit
	Delete

FIDO2 Token Find

Use /FIDO2/credential/find to search for registered AccessMatrix FIDO2 tokens. The AccessMatrix FIDO2 token contains a FIDO2 credential.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
serialNum_eq	String	Optional	Find by serial number.
UUID_eq	String	Optional	Find by token UUID.
batchUUID_eq	String	Optional	For tokens in batch with this UUID.
batchId_eq	String	Optional	For tokens in batch with this ID.

Parameter	Data Type	Required/ Optional	Description
groupUUID_eq	String	Optional	For tokens in group with this UUID.
groupId_eq	String	Optional	For tokens in group with this ID.
status_eq	String	Optional	For tokens with this status. Note: You may refer to the list of accepted values in Token States .
maxResult	String	Optional	Maximum number of results to return.
template	String	Optional	The template parameter is a string of template numbers separated by comma, eg. "1,2,3" that specifies which fields are to be returned. Individual field names can be passed to make the template more specific. Field names can also be combined with numbers, e.g. "1,batch,assignedUserIds". Note: You may refer the list of templates and field names in Templates .

REST Sample Code

FIDO2 Token Find API Call
URL: /FIDO2/credential/find

FIDO2 Token Find API Call

Request

```
JSON
{
  "sessionToken": "string",
  "serialNum_eq": "FYQnp_7Sh7ecbuJORv2K0gE8-fHYdTW8",
  "UUID_eq": "xNSaPL7QcUlHBrPYXDKaxKzSOH2h35HJ_jP7f2aWY8E",
  "batchUUID_eq": "jA9Pw",
  "batchId_eq": "fido2tokenbatch",
  "groupUUID_eq": "efP-",
  "groupId_eq": "fido2tokengroup",
  "status_eq": "Activated",
  "maxResults": "15",
  "template": "5,6"
}
```

Response

```
JSON
{
  "result": [
    {
      "serialNum": "dFvwi0ONXZqWshRkOkYVUr2yILCbZJ2o",
      "vendor": "FIDO2",
      "name": "FIDO2Credential",
      "objectClass": "Token",
      "reserved1": "WjZW7Kcunt",
      "attributes": {
        "FIDO2_RPIId": "localhost"
      },
      "model": "FIDO2",
      "id": "FIDO2-dFvwi0ONXZqWshRkOkYVUr2yILCbZJ2o",
      "UUID": "qBThUsbBaviN6EIbifIYecMzEjR2GfHhWznHZZnHLDA",
      "version": 1,
      "properties": [
        {
          "transports": "internal",
          "aagUID": "00000000-0000-0000-0000-000000000000",
          "registeredDate": "2024-11-12T08:01:52Z",
          "badVerificationCount": "0",
          "signatureCount": "0",
          "lastSuccAuth": "",
          "discoverable": "true",
          "appId": "FIDO2",

```

FIDO2 Token Find API Call

```
        "credentialId":  
        "dFvwi0ONXZqWshRkygUwJku01xuOkYVUr2yILCbZJ2o"  
    },  
    ],  
    "status": "Activated"  
},  
],  
"amProcessingTimeMillis": 37  
}
```

FIDO2 Token List

Use /FIDO2/credential/list to return a list of the user's registered AccessMatrix FIDO2 tokens. The AccessMatrix FIDO2 token contains a FIDO2 credential.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
userUUID	String	Required	UUID of user.
userId	String	Required	ID of user.
userSegmentId	String	Optional	ID of segment that contains the user. This is used in cases where the userId is not unique globally.
userStoreId	String	Optional	Store ID. This is used in cases where the user is from an external store.
template	String	Optional	The template parameter is a string of template numbers separated by comma, e.g. "1,2,3" that specifies which fields to be returned. Individual field names can be passed to make the template more specific. Field names can also be combined with numbers, e.g. "1,batch,assignedUserIds".

Parameter	Data Type	Required/ Optional	Description
			Note: You may refer the list of templates and field names in Templates .

REST Sample Code

FIDO2 Token List API Call
URL: /FIDO2/credential/list
Request
JSON <pre>{ "sessionToken": "string", "userUUID": "quduUZvWmv" "userId": "fido2user", "userStoreId": "ActiveDirectory", "userSegmentId": "SampleSegment1.1", "template": "5,6"}</pre>
Response
JSON <pre>{ "result": [{ "serialNum": "VjutPSl7ZuxIx2l4IXZMTuravuAkkU18", "vendor": "FIDO2", "name": "FIDO2Credential", "objectClass": "Token", "properties": [{ "transports": "internal", "aagUID": "00000000-0000-0000-0000-000000000000", "registeredDate": "2024-11-12T08:01:52Z", "badVerificationCount": "0", "signatureCount": "0", "lastSuccAuth": "", "discoverable": "true", "appId": "FIDO2", "credentialId": "dFvwi0ONXZqWshRkygUwJku0lxuOkYVUr2yILCbZJ2o" }], "reserved1": "1B9RqHGwAV", "attributes": {</pre>

FIDO2 Token List API Call
<pre> "FIDO2_RPId": "fido2.com" }, "model": "FIDO2", "id": "FIDO2-VjutPSl7ZuxIx2l4IXZMTuravuAkkU18", "UUID": "yCM3zL97HMKtEcKp6FwLWh42WDT86CiEbcZ9aGDgzJA", "version": 1, "status": "Activated" }], "amProcessingTimeMillis": 23 }</pre>

FIDO2 Token Modify

Use /FIDO2/credential/modify to modify the credential name stored in the AccessMatrix FIDO2 token.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
UUID	String	Required	Token UUID.
serialNumber	String	Required	Token serial number.
name	String	Optional	New token name to be modified to.

REST Sample Code

FIDO2 Token Modify API Call		
URL: /FIDO2/credential/modify		
Request <table> <tr> <th>JSON</th></tr> <tr> <td> <pre> { "sessionToken": "string", "UUID": "xNSaPL7QcU1HBrPYXDkaxKzSOH2h35HJ_jP7f2aWY8E", "serialNumber": "FYQnp 7Sh7ecbuJORv2K0gE8-fHYdTW8", </pre> </td></tr> </table>	JSON	<pre> { "sessionToken": "string", "UUID": "xNSaPL7QcU1HBrPYXDkaxKzSOH2h35HJ_jP7f2aWY8E", "serialNumber": "FYQnp 7Sh7ecbuJORv2K0gE8-fHYdTW8", </pre>
JSON		
<pre> { "sessionToken": "string", "UUID": "xNSaPL7QcU1HBrPYXDkaxKzSOH2h35HJ_jP7f2aWY8E", "serialNumber": "FYQnp 7Sh7ecbuJORv2K0gE8-fHYdTW8", </pre>		

FIDO2 Token Modify API Call
<pre>"name": "MySecurityKey1" }</pre>
Response
JSON
<pre>{ "result": { "responses": [] }, "amProcessingTimeMillis": 15 }</pre>

FIDO2 Token Change Status

Use /FIDO2/credential/changeStatus to change the status for a given AccessMatrix FIDO2 token.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
UUID	String	Required	Token UUID.
serialNumber	String	Required	Token serial number.
status	String	Required	New token status to change to. Note: You may refer to the list of accepted values in Token States .

REST Sample Code

FIDO2 Token Change Status API Call
URL: /FIDO2/credential/changeStatus
Request

FIDO2 Token Change Status API Call

JSON

```
{
  "dynamicAgentSession":true,
  "UUID":"xd02b08wPY1bj6px2doixySp63Umyy_WiQn8xrWSTFY",
  "serialNumber":"n3BqX2phQDL4Y5o0hzsnDXSrBV5BsqrEU",
  "status":"Deactivated"
}
```

Response

JSON

```
{
  "result": {
    "challenge": {
      "authenticationCode": -1,
      "params": {
        "tokenSerNum":
"n3BqX2phQDL4Y5o0hzsnDXSrBV5BsqrEU",
        "tokenStatus": "Deactivated",
        "vendorId": "FIDO2",
        "tokenUUID":
"xd02b08wPY1bj6px2doixySp63Umyy_WiQn8xrWSTFY",
        "tokenModel": "FIDO2",
        "oldTokenStatus": "Activated"
      }
    },
    "responses": [
      {}
    ]
  },
  "amProcessingTimeMillis": 148
}
```

FIDO2 Token Resync

Use FIDO2/credential/resync to resync the token. This will reset the FIDO2 token's bad verification count and optionally, set it to a new token status.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.

Parameter		Data Type	Required/ Optional	Description
UUID		String	Required	UUID of token.
serialNumber		String	Required	Serial number of token.
params		Object	Optional	
	newStatus	String	Optional	New status after token resync

REST Sample Code

FIDO2 Token Resync API Call	
URL: /FIDO2/credential/resync	
Request	
JSON <pre>{ "sessionToken": "", "UUID": "nQPQr_S5csjFzuiSRfJh8KM1cwrGpYOf2MrTjeXwp3E" "serialNumber": "lsEirHNKmgTx5BRvddZagJLqM_wFQS2k", "params": { "newStatus": "Activated" } }</pre>	
Response	
JSON <pre>{ "result": { "challenge": { "authenticationCode": -1, "params": { "tokenSerNum": "lsEirHNKmgTx5BRvddZagJLqM_wFQS2k", "tokenStatus": "Activated", "vendorId": "FIDO2", "tokenUUID": "nQPQr_S5csjFzuiSRfJh8KM1cwrGpYOf2MrTjeXwp3E", "tokenModel": "FIDO2",</pre>	

FIDO2 Token Resync API Call

```
        "oldTokenStatus": "Locked",
        "TokenMgtIsVerifyingAfterResync": false
    },
    "responses": [
        {}
    ],
    "amProcessingTimeMillis": 212
}
```

FIDO2 Token Delete

Use /FIDO2/credential/delete to delete the specified AccessMatrix FIDO2 token containing the FIDO2 credential. Once deleted, the credential will no longer be usable.

Parameter	Data Type	Required/Optional	Description
sessionToken	String	Required	User's session token or dynamic agent session.
UUID	String	Required	Token UUID.
serialNumber	String	Required	Token serial number.

REST Sample Code

FIDO2 Token Delete API Call

URL: /FIDO2/credential/delete

Request

JSON

```
{
  "sessionToken": "string",
  "UUID": "xNSaPL7QcU1HBrPYXDkaxKzSOH2h35HJ_jP7f2aWY8E",
```

FIDO2 Token Delete API Call

```
"serialNumber": "FYQnp_7Sh7ecbuJORv2K0gE8-fHYdTW8"
}
```

Response

JSON

```
{
  "result": {
    "responses": []
  },
  "amProcessingTimeMillis": 15
}
```

6.2. Authentication Service

The REST API Authentication Service is used for logins. A typical login process involves navigating through the authentication chains that are specified in a user's authentication realm.



Note: The Authentication Service can be used in place of the Token Service APIs for FIDO2 registration and assertion operations.

The following sections provide detailed information on performing authentication related operations on FIDO2 tokens:

- [Authentication Registration](#)
- [Authentication Assertion](#)

6.2.1. Authentication Registration

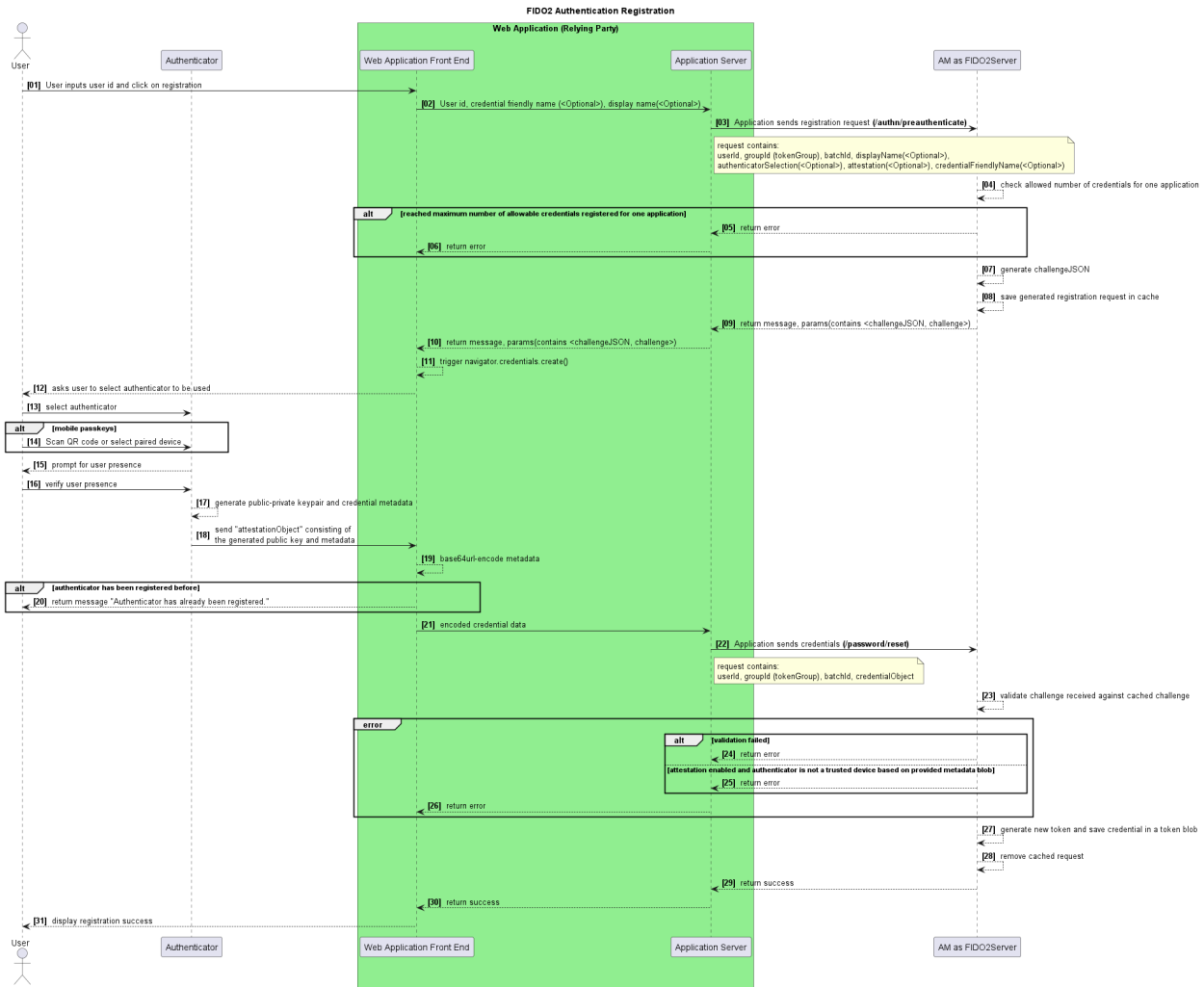
This section provides details on performing FIDO2 registration using the REST API Authentication Service. The registration process is initiated by calling the Authentication Service URL `/authn/preauthenticate`. To complete the registration, the `/password/reset` endpoint must subsequently be called. These APIs provide unified endpoints for performing authentication registration, allowing developers to use similar endpoints across both FIDO2 login modules and all other login modules (e.g. OATH, SMS OTP, etc).



Note: The REST API Authentication Service is an alternative to the REST API Token Service; it performs the same registration operation.

Sequence Diagram

The sequence diagram below shows the overall flow between the User, Authenticator, Relying Party (the Web Application Front End and its Application Server), and AccessMatrix (AM) as the FIDO2 Server. The Relying Party implementation is project-specific and beyond the scope of this document.



FIDO2 Authentication Registration Start

Use /authn/preauthenticate with the purpose field set to "registration" to start the FIDO2 registration process.

The response will contain a challengeJSON JSON string containing the various parameters generated by the FIDO2 server as well as a challenge string extracted from challengeJSON. These parameters in challengeJSON will be used by the client side to generate a credential for the authenticator.

Parameter	Data Type	Required / Optional	Description

Parameter		Data Type	Required / Optional	Description
loginModuleId		String	Required	The FIDO2 login module Id that the user is tagged with. Pre-set the FIDO2 login module Id in the application, if possible.
params		Object	Required	
	purpose	String	Required	Set to "registration".
	userId	String	Required	User's ID
	userStoreId	String	Optional	Store ID if user is from external store. If unspecified, AM will search for users in the default user store.
	segmentId	String	Optional	When user ID is not globally unique, segment must be specified.
	displayName	String	Optional	User-defined name for the user to be used for display purposes
	authenticatorSelection	Object	Optional	
	userVerification	String	Optional	Must be a value of 'preferred', 'discouraged' or 'required'. If not provided, value 'preferred' will be used
	requireResidentKey	Boolean	Optional	Either the requireResidentKey or residentKey fields can be specified. If both fields are specified, this field takes precedence. Default: false
	residentKey	String	Optional	Either the requireResidentKey or residentKey fields can be specified. If both fields are specified, the requireResidentKey field takes precedence. Must be a value of 'preferred', 'discouraged' or 'required'. Default: discouraged
	authenticatorAttachment	String	Optional	Must be a value of 'platform' or 'cross-platform'. If not provided, this field will not be used

Parameter		Data Type	Required / Optional	Description
	attestation	String	Optional	Must be a value of 'direct', 'indirect' or 'none'. If not provided, value 'none' will be used.
	credentialFriendlyName	String	Optional	A user defined name to identify the credential. If not provided, will use 'FIDO2Credential'.

REST Sample Code

FIDO2 Authentication Registration Start API Call	
URL: /authn/preauthenticate	
Request JSON <pre> { "loginModuleId": "fido2lm", "params": { "purpose": "registration", "userId": "fido2user", "userStoreId": "ActiveDirectory", "segmentId": "SampleSegment1.1", "displayName": "John", "authenticatorSelection": { "userVerification": "preferred", "requireResidentKey": "false", "authenticatorAttachment": "cross-platform" }, "attestation": "direct", "credentialFriendlyName": "MyCredential" } } </pre>	
Response JSON <pre> { "result": { "challengeToken": "The challenge token is not applicable for FIDO2 token implementation.", "authenticationCode": 0, "message": "Successful Preauthentication.", "params": { "challengeJSON": "", "challenge": "" } } } </pre>	

FIDO2 Authentication Registration Start API Call
<pre> { } } </pre>

FIDO2 Authentication Registration Finish (/password/reset)

Use /password/reset to finish the FIDO2 registration process.

If the registration is successful, the response will contain a field authenticated with the value "true".

Parameter	Data Type	Required / Optional	Description
password	String	Required	Contains the challengeJSON JSON string returned by calling the /authn/preauthenticate API.
loginModuleId	String	Required	The FIDO2 login module Id that the user is tagged with. Pre-set the FIDO2 login module Id in the application, if possible.
userId	String	Required	User's ID
userStoreId	String	Optional	Store ID if user is from external store. If unspecified, AM will search for users in the default user store.
params	Object	Required	
segmentId	String	Optional	When user ID is not globally unique, segment must be specified.

REST Sample Code

FIDO2 Authentication Registration Finish API Call
URL: /password/reset
Request
JSON
<pre> { "password" : "", "loginModuleId": "fido2lm", "userId": "fido2user", </pre>

FIDO2 Authentication Registration Finish API Call

```
"userStoreId": "ActiveDirectory"
"params": {
  "segmentId": "SampleSegment1.1"
}
}
```

Response

JSON

```
{
  "result": {
    "authenticated": true,
    "sessionIID": "",
    "lastAccessTime": 10000000000000,
    "sessionFlags": 0,
    "sessionToken": "",
    "responses": {},
    "idleTimeOutInSecs": 30000,
    "userUUID": "",
    "loginModuleStates": [],
    "serverId": "",
    "userId": ""
  },
  "amProcessingTimeMillis": 208
}
```

6.2.2. Authentication Assertion

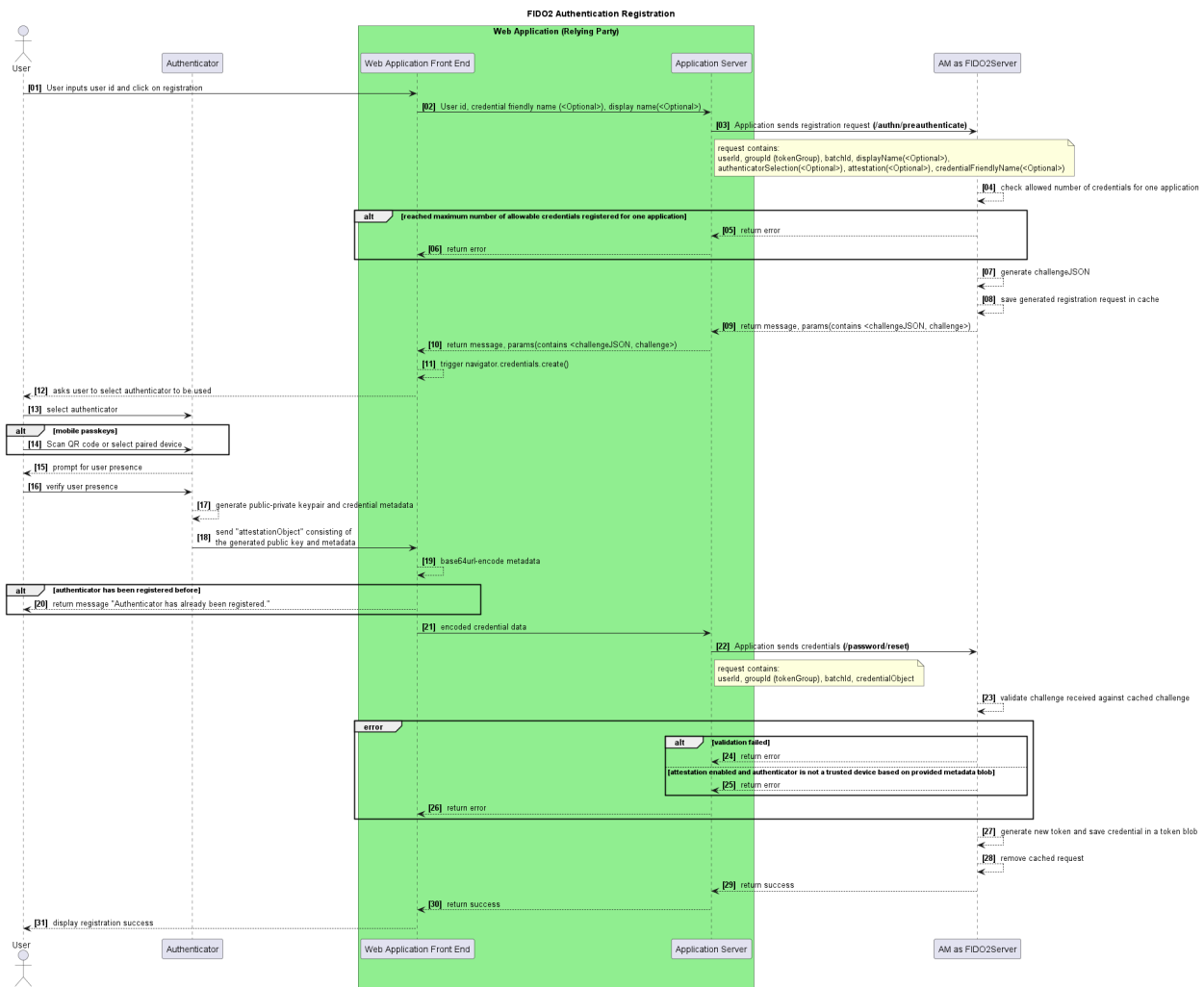
This section provides details on performing FIDO2 authentication using the REST API Authentication service. The assertion process is initiated by calling the Authentication Service endpoint `/authn/preauthenticate`. To complete the assertion, the `/password/verify` or `/authn/login` endpoints must subsequently be called. These APIs provide unified endpoints for performing authentication assertion, allowing developers to use similar endpoints across both FIDO2 login modules and all other login modules (e.g. OATH, SMS OTP, etc).



Note: The REST API Authentication Service is an alternative to the REST API Token Service; it performs the same assertion operation.

Sequence Diagram

The sequence diagram below shows the overall flow between the User, Authenticator, Relying Party (the Web Application Front End and its Application Server), and AccessMatrix (AM) as the FIDO2 Server. The Relying Party implementation is project-specific and beyond the scope of this document.



FIDO2 Authentication Assertion Start

Use `/authn/preauthenticate` with the purpose field set to "authentication" to start the FIDO2 authentication process.

The response will contain a set of parameters that includes all the metadata required to authenticate the key.

Parameter	Data Type	Required / Optional	Description
loginModuleId	String	Required	The FIDO2 login module Id that the user is tagged with. Pre-set the FIDO2 login module Id in the application, if possible.
params	Object	Required	

Parameter	Data Type	Required / Optional	Description
purpose	String	Required	Set to "authentication".
userId	String	Optional	User's ID
userStoreId	String	Optional	Store ID if user is from external store. If unspecified, AM will search for users in the default user store.
segmentId	String	Optional	When user ID is not globally unique, segment must be specified.
userVerification	String	Optional	Must be a value of 'preferred', 'discouraged' or 'required'. If not provided, value 'preferred' will be used

REST Sample Code

FIDO2 Authentication Assertion Start API Call	
URL: /authn/preauthenticate	
Request	
JSON	<pre>{ "loginModuleId": "fido2lm", "params": { "purpose": "authentication", "userId": "fido2user", "userStoreId": "ActiveDirectory", "segmentId": "SampleSegment1.1", "userVerification" : "discouraged" } }</pre>
Response	
JSON	<pre>{ "result": { "challengeToken": "The challenge token is not applicable for FIDO2 token implementation.", "authenticationCode": 0, "message": "Successful Preauthentication.", "params": { "challengeJSON": "", "challenge": "" } } }</pre>

FIDO2 Authentication Assertion Start API Call
<pre> } } </pre>

FIDO2 Authentication Assertion Finish (/password/verify)



Note: For usernameless use cases, the web application back end should resolve the userHandle and call AM5 to find the user before calling /password/verify.

Use /password/verify to finish the authentication process.

If the authentication is successful, the response will contain a field authenticated with the value "true".

Parameter	Data Type	Required/Optional	Description
loginModuleId	String	Required	The FIDO2 login module Id that the user is tagged with. Pre-set the FIDO2 login module Id in the application, if possible.
userId	String	Required	User's ID
userStoreId	String	Optional	Store ID if user is from external store. If unspecified, AM will search for users in the default user store.
segmentId	String	Optional	When user ID is not globally unique, segment must be specified.
password	String	Required	The result of the successfully resolved promise from the RP client-side in the JSON string format, to be returned to AM to be verified.

REST Sample Code

FIDO2 Authentication Assertion Finish API Call
URL: /password/verify
Request
JSON
<pre> { "loginModuleId": "fido2lm", </pre>

FIDO2 Authentication Assertion Finish API Call

```
{
  "userId": "fido2user",
  "userStoreId": "ActiveDirectory"
  "segmentId": "SampleSegment1.1",
  "password" : ""
}
```

Response

JSON

```
{
  "result": {
    "responses": {},
    "loginModuleStates": [{
      "authenticated": true,
      "canChangePassword": false,
      "id": "fido2lm",
      "forceChangePassword": true,
      "loginModuleHandlerClass": "FIDO2TokenLogin",
      "forceChangePasswordReason": 1
    }]
  },
  "amProcessingTimeMillis": 342
}
```

FIDO2 Authentication Assertion Finish (/authn/login)

Call /authn/login to finish the authentication process.

If the authentication is successful, the response will contain a field `authenticated` with the value "true".

Parameter	Data Type	Required/ Optional	Description
realmId	String	Required	The FIDO2 realm Id that the user is tagged with. Pre-set the FIDO2 login module Id in the application, if possible.
userId	String	Required	User's ID.
segmentId	String	Optional	When user ID is not globally unique, segment must be specified.
password	String	Required	The result of the successfully resolved promise from the RP client-side in the JSON string format, to be returned to AM to be verified.

REST Sample Code

FIDO2 Authentication Assertion Finish API Call

URL: /authn/login

Request

JSON

```
{
  "realmId": "fido2ar",
  "userId": "fido2user",
  "segmentId": "SampleSegment1.1",
  "password" : ""
}
```

Response

JSON

```
{
  "result": {
    "attr.LT": "1705340244672",
    "authenticated": true,
    "sessionIID": "",
    "lastAccessTime": 1705310244744,
    "realmId": "fido2ar",
    "sessionFlags": 1,
    "serverId": "",
    "userId": "fido2user",
    "attr.IT": "30000",
    "expiresAt": 1705340244,
    "realm.lastLoginSuccessTime": 1705310160000,
    "loginTime": 1705310244000,
    "sessionToken": "",
    "responses": {},
    "idleTimeOutInSecs": 30000,
    "userUUID": "",
    "loginModuleStates": [{
      "authenticated": true,
      "canChangePassword": false,
      "id": "fido2lm",
      "forceChangePassword": true,
      "loginModuleHandlerClass": "fido2lm",
      "forceChangePasswordReason": 3
    }],
    "attr.regenerated": "1705310244745"
  },
  "amProcessingTimeMillis": 74
}
```

7. UAS FIDO2 Token Housekeeping Policy

This section includes information on how to configure housekeeping tasks for FIDO2 tokens.

Housekeeping Policy for FIDO2 Token

The housekeeping policy contains rules on housekeeping FIDO2 token data. To configure the housekeeping policy, navigate to **Administration Dashboard > System > Housekeeping** and click **Housekeeping Policy** on the left pane. Select the **UAS** tab and click the **Edit** button. Save the changes once modifications have been made.

For a list of housekeeping attributes applicable, refer to the following:

Attributes	Description
UAS Tab	
Tokens	
Purge non-SMS token if it is in a certain status for longer than the specified period	Specifies how long non-SMS tokens can remain in a certain status before they are purged. Define the status for the token, i.e. "Battery Expired", and its period in months, i.e. "24", and then add the status and its period to the list. Note: Period (Months) should be between 1 and 1200

Housekeeping FIDO2 Challenge Stored in Database

The FIDO2 challenge is a string of randomly generated cryptographic data that is used as part of the authentication process. By default, this will be cached for a short duration. However, the FIDO2 challenge can also be configured for database storage. In this configuration, housekeeping tasks must be executed to remove the FIDO2 challenge from the database regularly.

To enable the FIDO2 challenge to be stored in the database and configure the housekeeping tasks, perform the following:

1. Navigate to `<ACMATRIX_ROOT>\WEB-INF\classes\` and open the **amsystem.properties** file.
 2. Set the `am.fido2.challenge.persistence` parameter to "true" to store the FIDO2 challenge in the database.
-

-
3. Configure the housekeeping interval and expiry for the FIDO2 challenge as required. For example:

Properties (.properties files)

```
# == AM FIDO2 Challenge ==  
#If enabled, challenge object will be stored in DB  
instead.  
am.fido2.challenge.persistence=true  
#Only effective if challenge persistence is enabled.  
Sets the interval for housekeeping in minutes, default  
is 60  
am.fido2.challenge.housekeeping.interval=60  
#Sets the expiry of the challenge in minutes, default  
is 5  
am.fido2.challenge.expiry = 5
```

4. Save the changes.

For a list of housekeeping attributes applicable, refer to [AccessMatrix Common Deployment Guide - AM FIDO2 Challenge](#).

8. UAS FIDO2 Token Reports

The AccessMatrix UAS server audits important user and system events within the FIDO2 token registration, authentication, and deregistration flows. Currently, the following audit events are supported:

Event Type	Description
FIDO2StartRegistration	This audit event is raised after the FIDO2 registration process is initiated.
FIDO2FinishRegistration	This audit event is raised after the FIDO2 registration process is completed.
FIDO2StartAssertion	This audit event is raised after the FIDO2 authentication process is initiated.
FIDO2FinishAssertion	This audit event is raised after the FIDO2 authentication process is completed.



Note: Refer to [UAS Administration Guide - Token Activities Report](#) for more information on token audit reports.

The above events can be viewed by generating either an Events Report or Token Activities Report in the Admin Console:

1. Navigate to either:
 - a. Events Report - **Report Dashboard > System & Admin > Historical > Events.**
 - b. Token Activities Report - **Report Dashboard > Token & SMS > Historical > Token Activities.**
2. For **Event Category**, ensure **All Event Categories** is selected.
 - a. Alternatively, click **Select Event Category** and select **Token**.
3. Input the report criteria, select the report format (i.e. "HTML", "PDF" or "CSV") and click the **Generate** button.

9. Appendices

This section contains additional information that complements this guide.

Templates

These templates are used as a parameter in the following API calls:

- /FIDO2/registration/finish
- /FIDO2/credential/find
- /FIDO2/credential/list

Template	Fields	Remark
None	UUID, vendor, serialNum, model	These basic fields are always returned.
1	groups	Token Groups that the token belongs to.
2	batch	Token batch that the token belongs to.
3	assignedUserUUIDs	
4	assignedUserIds	
5	properties	Token blobs.
6	attributes	Custom attributes.
